

การพัฒนาระบบเว็บแอปพลิเคชันแบ่งปันของเหลือใช้ โดยใช้สถาปัตยกรรม REST API ร่วมกับฐานข้อมูล MongoDB

Development of a Web-Based Sharing Platform for Unused Household Items Using REST API Architecture with MongoDB Database

โดย

673380284-4 นายพงศ์อนันต์ วงศ์ศรี

673380050-9 นายปิยพนธ์ แก้วเก็บคำ

673380517-7 นายอภิวิชญ์ เอกะ

673380285-2 นายพิสิษฐ์ ทรัพย์อุดมโชติ

อาจารย์ที่ปรึกษา

รองศาสตราจารย์ ดร.งามนิจ อาจอินทร์

รายงานฉบับนี้เป็นส่วนหนึ่งของการศึกษา

วิชา CP363205 ภาษาอนุกรมข้อมูลและการประยุกต์

หลักสูตรวิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการคอมพิวเตอร์

วิทยาลัยการคอมพิวเตอร์

มหาวิทยาลัยขอนแก่น

ปีการศึกษา 2568

คำนำ

รายงานฉบับนี้จัดทำขึ้นเพื่อประกอบรายวิชา CP363205 ภาษาอนุกรมข้อมูลและการประยุกต์ โดยมีวัตถุประสงค์เพื่อนำเสนอรายละเอียดการพัฒนาโครงงาน “การพัฒนาระบบเว็บแอปพลิเคชันแบ่งปันของเหลือใช้โดยใช้สถาปัตยกรรม REST API ร่วมกับฐานข้อมูล MongoDB” ตั้งแต่ความเป็นมาและที่มาของปัญหา วัตถุประสงค์ ทฤษฎีและเครื่องมือที่ใช้ โครงสร้างข้อมูล JSON โครงสร้าง API ขอบเขตและฟังก์ชันของระบบ ไปจนถึงผลการดำเนินงานและผลการทดสอบระบบจริง

เนื้อหาภายในเล่มแบ่งออกเป็น 5 บท ได้แก่ บทนำ วรรณกรรมและงานวิจัยที่เกี่ยวข้อง อุปกรณ์และวิธีดำเนินการ ผลการดำเนินงาน และสรุปผลการดำเนินงาน อภิปรายผล และข้อเสนอแนะ พร้อมภาคผนวกแสดงตัวอย่างการเรียกใช้ API โครงสร้างโปรเจกต์ ตาราง Endpoint ทั้งหมดของระบบ และภาพหน้าจอการทำงานจริงของระบบ

คณะผู้จัดทำหวังเป็นอย่างยิ่งว่ารายงานฉบับนี้จะเป็นประโยชน์ต่อผู้ที่สนใจศึกษาการพัฒนาเว็บแอปพลิเคชันด้วยสถาปัตยกรรม REST API และฐานข้อมูล MongoDB หากมีข้อผิดพลาดประการใด คณะผู้จัดทำขอน้อมรับไว้เพื่อปรับปรุงแก้ไขต่อไป

คณะผู้จัดทำ

สารบัญ

คำนำ . . . . . ก
สารบัญ . . . . . ข
สารบัญตาราง . . . . . ง
สารบัญภาพ . . . . . จ

หน้าที่

บทที่ 1 บทนำ 1
1.1 ความเป็นมาและความสำคัญของปัญหา 1
1.2 วัตถุประสงค์ของโครงงาน 1
1.3 ประโยชน์ที่คาดว่าจะได้รับ 2
1.4 ขอบเขตและข้อจำกัดของโครงงาน 2
1.4.1 ขอบเขต 2
1.4.2 ฟังก์ชันของระบบแบ่งตามผู้ใช้งาน 3
1.4.3 ข้อจำกัด 3
1.5 ระยะเวลาในการดำเนินการ 3

บทที่ 2 วรรณกรรมและงานวิจัยที่เกี่ยวข้อง 4
2.1 แนวคิดเศรษฐกิจแบ่งปัน 4
2.2 งานวิจัยเกี่ยวกับระบบแบ่งปันและซื้อขายของมือสอง 4
2.3 สถาปัตยกรรม REST API 5
2.4 ฐานข้อมูล MongoDB และการค้นหาเชิงภูมิศาสตร์ 5
2.5 การยืนยันตัวตนด้วย Session 5
2.6 เอกสาร API อัตโนมัติด้วย OpenAPI และ Swagger 5
2.7 ระบบเทมเพลตฝั่งเซิร์ฟเวอร์ EJS 6

บทที่ 3 อุปกรณ์และวิธีดำเนินการ 7

3.1 อุปกรณ์และซอฟต์แวร์ . . . 7
3.2 การออกแบบและพัฒนาระบบ . . . 7
3.2.1 สถาปัตยกรรมระบบ . . . 7
3.2.2 การพัฒนาหน้าจอผู้ใช้ด้วย EJS . . . 8
3.2.3 การออกแบบฐานข้อมูล . . . 8
3.2.4 พจนานุกรมข้อมูล (Data Dictionary) . . . 9
3.2.5 โครงสร้างข้อมูล JSON (JSON Map Structure) . . . 9
3.2.6 โครงสร้าง API (API Map Structure) . . . 14
3.2.7 กระบวนการขอรับและส่งมอบของ . . . 15
3.3 ข้อมูลจำเพาะของระบบที่ใช้ทดสอบและนำเสนอ (System Specifications) . . . 15
3.4 การทดสอบระบบ . . . 15

บทที่ 4 ผลการดำเนินงาน . . . 17
4.1 ผลการพัฒนาระบบ . . . 17
4.2 การตรวจพบข้อบกพร่องและการพัฒนาเพิ่มเติม . . . 17
4.3 ผลการทดสอบระบบ . . . 19

บทที่ 5 สรุปผลการดำเนินงาน อภิปรายผล และข้อเสนอแนะ . . . 21
5.1 สรุปผลการดำเนินงาน . . . 21
5.2 อภิปรายผล . . . 21
5.3 ข้อเสนอแนะสำหรับการวิจัยครั้งต่อไป . . . 22
5.3.1 ข้อเสนอแนะสำหรับการนำผลงานไปใช้ . . . 22
5.3.2 ข้อเสนอแนะในการศึกษาครั้งถัดไป . . . 22

บรรณานุกรม . . . 23

ภาคผนวก A ตัวอย่างการเรียกใช้ API . . . 24

ภาคผนวก B โครงสร้างโปรเจกต์และวิธีการรันระบบ . . . 26

ภาคผนวก C ตาราง Endpoint ของระบบทั้งหมด . . . 28

ภาคผนวก D ภาพหน้าจอระบบ . . . 31

สารบัญตาราง

ตารางที่		หน้าที่
1	คอลเลกชันหลักของระบบและหน้าที่	9
2	พจนานุกรมข้อมูลคอลเลกชัน User	10
3	พจนานุกรมข้อมูลคอลเลกชัน Category	10
4	พจนานุกรมข้อมูลคอลเลกชัน Listing	11
5	พจนานุกรมข้อมูลคอลเลกชัน Reservation	12
6	พจนานุกรมข้อมูลคอลเลกชัน Report	13
7	ข้อมูลจำเพาะของระบบที่ใช้ทดสอบและนำเสนอ	16
8	กลุ่ม API และหน้าที่หลักของแต่ละกลุ่ม	17
9	สรุปข้อบกพร่องที่ตรวจพบและการแก้ไข/พัฒนาเพิ่มเติม	18
10	ผลการทดสอบระบบในแต่ละสถานการณ์	20
11	รายการ Endpoint ทั้งหมดของระบบ	28

สารบัญภาพ

ภาพที่		หน้าที่
1	สถาปัตยกรรมของระบบ	8
2	สถานะของคำขอรับของ (Reservation)	15
3	หน้าแรกของระบบ (มุมมองผู้เยี่ยมชม)	31
4	หน้าค้นหาประกาศ พร้อมตัวกรองหมวดหมู่ สภาพ และพื้นที่	32
5	หน้ารายละเอียดประกาศ (มุมมองผู้เยี่ยมชม — ข้อมูลนัดรับถูกปกปิดจนกว่าจะได้รับอนุมัติ)	32
6	หน้าเข้าสู่ระบบ/สมัครสมาชิก	33
7	หน้าภาพรวมส่วนตัว (Dashboard) ของสมาชิก	33
8	ฟอร์มลงประกาศใหม่ พร้อม Dropdown จังหวัด/อำเภอ/ตำบลแบบลำดับขั้น และปุ่มอัปโหลดรูปภาพ	34
9	หน้าประกาศของฉัน แสดงสถานะและจำนวนคำขอที่รอพิจารณาของแต่ละประกาศ	34
10	หน้าแก้ไขประกาศ แสดงสถานะถูกล็อก (Strict Edit Lock) เนื่องจากมีผู้ยื่นคำขอรับแล้ว	35
11	หน้าจัดการคำขอที่ได้รับ สำหรับเจ้าของประกาศ	35
12	หน้าคำขอของฉัน แสดงข้อมูลนัดรับที่ปลดล็อกแล้วหลังได้รับอนุมัติ	36
13	หน้าโปรไฟล์ของฉัน พร้อม Dropdown ที่อยู่แบบลำดับขั้น	36
14	หน้าเปลี่ยนรหัสผ่าน	37
15	หน้าวิธีใช้งาน (ขั้นตอนการทำงานและคำถามที่พบบ่อยแบบ Accordion)	37
16	หน้าภาพรวมระบบสำหรับผู้ดูแลระบบ (Admin Back-office)	38
17	หน้าจัดการประกาศทั้งหมดของผู้ดูแลระบบ (ดูได้ทุกสถานะรวมถึงที่ยกเลิก/หมดอายุ)	38
18	หน้าจัดการรายงานปัญหาของผู้ดูแลระบบ	39
19	หน้าจัดการผู้ใช้ของผู้ดูแลระบบ (ระงับ/เปิดการใช้งานบัญชี)	39
20	หน้าจัดการหมวดหมู่ของผู้ดูแลระบบ พร้อมจำนวนประกาศที่ใช้งานแต่ละหมวดหมู่	40
21	หน้าแสดงข้อผิดพลาด 404 เมื่อเข้าถึงเส้นทางที่ไม่มีอยู่จริง	40
22	เอกสาร API ที่สร้างอัตโนมัติผ่าน Swagger UI	41

บทที่ 1

บทนำ

1.1 ความเป็นมาและความสำคัญของปัญหา

ในปัจจุบันการบริโภคสินค้าของคนไทยเพิ่มขึ้นอย่างต่อเนื่อง ส่งผลให้เกิดสิ่งของที่ยังใช้งานได้แต่ไม่ได้รับการใช้งานจำนวนมากในแต่ละครัวเรือน เช่น เฟอร์นิเจอร์ที่เหลือจากการย้ายที่พัก เครื่องใช้ไฟฟ้าที่ซื้อเครื่องใหม่มาทดแทน เสื้อผ้าที่ใส่ไม่ได้ขนาด และของใช้ในบ้านที่ซื้อมาเก็บไว้แต่ไม่ได้ใช้ โดยปกติเจ้าของสิ่งของเหล่านี้มักเลือกทิ้งลงถังขยะ ซึ่งนอกจากจะเป็นการสูญเปล่าทางเศรษฐกิจแล้ว ยังเพิ่มภาระให้ระบบการกำจัดขยะของชุมชนและส่งผลกระทบต่อสิ่งแวดล้อมโดยตรง

อีกทางเลือกหนึ่งที่นิยมใช้กันคือการฝากขายหรือแจกตามกลุ่มซื้อขายออนไลน์ทั่วไป แต่กลุ่มดังกล่าวถูกออกแบบมาสำหรับการซื้อขายเพื่อกำไรเป็นหลัก ผู้ใช้ที่ต้องการแบ่งปันสิ่งของจึงต้องผ่านขั้นตอนยุ่งยาก เช่น การต่อรองราคา การติดต่อสื่อสารที่กระจัดกระจายหลายช่องทาง และไม่มีระบบติดตามสถานะของรายการว่ามอบของแล้วหรือยัง อีกทั้งยังเกิดปัญหาความไม่น่าเชื่อถือของผู้ขายบางราย ทำให้ผู้ใช้ทั่วไปขาดความมั่นใจในการนำสิ่งของมาแบ่งปันกัน

แนวคิดเศรษฐกิจแบ่งปัน (Sharing Economy) คือรูปแบบทางเศรษฐกิจที่เน้นการใช้ทรัพยากรร่วมกันโดยไม่จำเป็นต้องเป็นเจ้าของถาวร ซึ่งได้รับความนิยมเพิ่มขึ้นอย่างมากในยุคดิจิทัล เนื่องจากแพลตฟอร์มออนไลน์ช่วยลดต้นทุนในการเชื่อมต่อระหว่างผู้ให้และผู้รับ ทำให้การแบ่งปันสิ่งของเกิดขึ้นได้ง่ายและมีประสิทธิภาพมากขึ้น ในต่างประเทศมีแพลตฟอร์มแบ่งปันสิ่งของมือสองจำนวนมาก ขณะที่ในประเทศไทยแพลตฟอร์มลักษณะนี้ที่เน้นการแบ่งปันโดยไม่มีการซื้อขายมีอยู่อย่างจำกัด

จากเหตุผลดังกล่าว คณะผู้จัดทำจึงเห็นว่าควรพัฒนาระบบเว็บแอปพลิเคชันแบ่งปันของเหลือใช้ที่มีกระบวนการทำงานครบวงจร ทั้งการลงประกาศ การขอรับหรือจองของ การอนุมัติโดยเจ้าของประกาศ และการปิดรายการเมื่อส่งมอบของแล้ว โดยออกแบบระบบให้เป็น REST API ที่แยกส่วนการทำงานออกจากหน้าจอผู้ใช้อย่างชัดเจน เพื่อให้สามารถนำไปเชื่อมต่อกับเว็บไซต์หรือแอปพลิเคชันมือถือได้ในอนาคต รวมถึงรองรับการค้นหาของใกล้ตัวตามระยะทางด้วยข้อมูลตำแหน่งภูมิศาสตร์ ซึ่งช่วยให้ผู้ใช้สามารถมอบรับของกันได้สะดวกโดยไม่ต้องเดินทางไกล

1.2 วัตถุประสงค์ของโครงงาน

1.2.1 เพื่อพัฒนาระบบลงประกาศและแบ่งปันของเหลือใช้ที่ผู้ใช้สามารถลงประกาศได้โดยไม่มีค่าใช้จ่าย

1.2.2 เพื่อพัฒนาระบบขอรับ จอง อนุมัติ และปิดรายการเมื่อส่งมอบของแล้วอย่างเป็นระบบ พร้อมป้องกันปัญหาการจองซ้ำและการอนุมัติเกินจำนวนของที่มีอยู่

1.2.3 เพื่อออกแบบ REST API พร้อมเอกสารประกอบอัตโนมัติ ที่สามารถนำไปเชื่อมต่อกับเว็บไซต์หรือแอปพลิเคชันอื่นได้ในอนาคต

1.3 ประโยชน์ที่คาดว่าจะได้รับ

1.3.1 ช่วยลดปริมาณขยะจากสิ่งของที่ยังใช้งานได้ โดยให้ผู้ใช้สามารถส่งต่อของเหลือใช้ถึงผู้ที่ต้องการใช้จริงได้โดยตรง

1.3.2 ผู้ใช้สามารถค้นหาของแบ่งปันในบริเวณใกล้ตัวตามระยะทางที่กำหนด ทำให้การนัดรับของสะดวกและประหยัดเวลา

1.3.3 ระบบติดตามสถานะคำขอรับอย่างชัดเจนตั้งแต่การขอรับ อนุมัติ จนถึงการส่งมอบ ทำให้ทั้งสองฝ่ายมั่นใจในรายการ

1.3.4 ผู้จัดทำได้รับประสบการณ์การออกแบบสถาปัตยกรรมระบบ การออกแบบฐานข้อมูล การพัฒนา REST API และการทดสอบระบบอย่างเป็นขั้นตอน

1.3.5 ระบบ ที่ พัฒนา ขึ้น มี เอกสาร API อัตโนมัติ สามารถ ต่อย อด พัฒนา ส่วน หน้า จอ ผู้ ใช้ หรือแอปพลิเคชันมือถือต่อได้โดยไม่ต้องเริ่มออกแบบใหม่

1.4 ขอบเขตและข้อจำกัดของโครงงาน

1.4.1 ขอบเขต

โครงงานนี้มุ่งเน้นการพัฒนาส่วนของ REST API สำหรับระบบแบ่งปันของเหลือใช้ ครอบคลุมการทำงานหลัก 3 เรื่อง ได้แก่ 1) ผู้ใช้สามารถลงทะเบียน เข้าสู่ระบบ และลงประกาศของเหลือใช้พร้อมระบุจำนวน สภาพของ วิธีการรับของ และตำแหน่งที่ตั้ง โดยเลือกจังหวัด อำเภอ และตำบลจากรายการข้อมูลภูมิศาสตร์ของประเทศไทย และอัปโหลดรูปภาพประกอบประกาศได้ทั้งจากไฟล์ในเครื่องหรือระบุเป็นลิงก์ URL 2) ผู้ใช้อื่นสามารถค้นหาประกาศด้วยหมวดหมู่ คำค้นหา และระยะทาง พร้อมยื่นคำขอรับของ และ 3) เจ้าของประกาศสามารถอนุมัติ ปฏิเสธ และยืนยันการส่งมอบเพื่อปิดรายการ รวมถึงระบบรายงานประกาศไม่เหมาะสมและระบบหลังบ้านสำหรับผู้ดูแลระบบ (Admin Back-office) สำหรับจัดการผู้ใช้ ประกาศ หมวดหมู่ และรายงานปัญหา ทั้งนี้ระบบยังพัฒนาหน้าจอผู้ใช้ (User Interface) ด้วยเทมเพลต EJS ฝั่งเซิร์ฟเวอร์ร่วมกับไลบรารี Semantic UI ครบทั้ง 19 เส้นทางหน้าเว็บ ครอบคลุมหน้ารวมประกาศ รายละเอียดประกาศ ฟอร์มลงประกาศ ฟอร์มยืนยันตัวตน และหน้าจัดการระบบของผู้ดูแลระบบ

1.4.2 ฟังก์ชันของระบบแบ่งตามผู้ใช้งาน

เพื่อให้เห็นขอบเขตของงานอย่างชัดเจน ฟังก์ชันของระบบแบ่งตามประเภทผู้ใช้งานได้ดังนี้

1.4.1 **ผู้เยี่ยมชม (Guest)** – ดูรายการประกาศและค้นหาประกาศได้ (เห็นเฉพาะพื้นที่โดยประมาณ ไม่เห็นเบอร์โทรหรือที่อยู่ละเอียด) สมัครสมาชิกและเข้าสู่ระบบ

1.4.2 **สมาชิก/ผู้ขอรับของ (User)** – ทำทุกอย่างที่ผู้เยี่ยมชมทำได้ เพิ่มเติมคือ ยื่นคำขอรับของ ยกเลิกคำขอของตนเอง ติดตามสถานะคำขอ ดูการแจ้งเตือน และแก้ไขข้อมูลส่วนตัว/รหัสผ่าน

1.4.3 **เจ้าของประกาศ (Owner)** – ลงประกาศใหม่ แก้ไขหรือยกเลิกประกาศของตนเอง (เมื่อยังไม่มีผู้ยื่นคำขอ) ดูรายชื่อผู้ยื่นคำขอของแต่ละประกาศ อนุมัติ ปฏิเสธ และยืนยันการส่งมอบของแต่ละคำขอ

1.4.4 **ผู้ดูแลระบบ (Admin)** – จัดการหมวดหมู่สิ่งของ (เพิ่ม แก้ไข ปิดใช้งาน) จัดการผู้ใช้ (ระงับ/ยกเลิกระงับบัญชีพร้อมเหตุผล) ตรวจสอบและจัดการรายงานประกาศไม่เหมาะสม ยกเลิกประกาศที่ไม่เหมาะสม และดูสรุปภาพรวมระบบ โดยไม่สามารถแก้ไขเนื้อหาประกาศหรืออนุมัติ/ปฏิเสธคำขอแทนเจ้าของประกาศได้

1.4.3 ข้อจำกัด

ระบบพัฒนาหน้าจอผู้ใช้ในรูปแบบเทมเพลตฝั่งเซิร์ฟเวอร์ (Server-Side Rendering) ด้วย EJS ซึ่งยังไม่รวมการทำงานแบบฝั่งลูกข่ายที่ซับซ้อน เช่น Single Page Application ระบบมีการแจ้งเตือนภายในแอปพลิเคชันผ่านสัญลักษณ์แจ้งเตือน (Notification Badge) แล้ว แต่ยังไม่มีการแจ้งเตือนแบบเรียลไทม์ผ่านช่องทางภายนอก เช่น อีเมลหรือแอปพลิเคชันส่งข้อความ เมื่อมีผู้ยื่นคำขอรับของ

1.5 ระยะเวลาในการดำเนินการ

สิงหาคม 2568 – ตุลาคม 2568

บทที่ 2

วรรณกรรมและงานวิจัยที่เกี่ยวข้อง

ผู้จัดทำโครงงานได้ศึกษาค้นคว้าทฤษฎีและเอกสารที่เกี่ยวข้องดังนี้

1. แนวคิดเศรษฐกิจแบ่งปัน (Sharing Economy)
2. งานวิจัยเกี่ยวกับระบบแบ่งปันและซื้อขายของมือสอง
3. สถาปัตยกรรม REST API
4. ฐานข้อมูล MongoDB และการค้นหาเชิงภูมิศาสตร์
5. การยืนยันตัวตนด้วย Session
6. เอกสาร API อัตโนมัติด้วย OpenAPI และ Swagger

2.1 แนวคิดเศรษฐกิจแบ่งปัน

เศรษฐกิจแบ่งปัน (Sharing Economy) คือรูปแบบเศรษฐกิจที่อิงจากการให้บุคคลได้รับสิทธิในการใช้สินทรัพย์ที่ไม่ได้ใช้งานของกันและกัน โดยมักอาศัยแพลตฟอร์มออนไลน์เป็นตัวเชื่อมต่อ การศึกษาของ Hamari, Sjöklint และ Ukkonen (2016) พบว่าแรงจูงใจหลักของผู้ที่เข้าร่วมการแบ่งปันทรัพยากรร่วมกัน คือความยั่งยืนทางเศรษฐกิจ การที่ผู้ใช้เห็นว่าเป็นกิจกรรมที่ดี และความสนุกในการใช้งาน ซึ่งแสดงว่าแพลตฟอร์มที่ออกแบบให้ใช้งานง่ายและเห็นประโยชน์ชัดเจนจะช่วยให้ผู้ใช้เข้าร่วมมากขึ้น งานของ Botsman และ Rogers (2010) ยังชี้ว่าหัวใจสำคัญของเศรษฐกิจแบ่งปันคือระบบความเชื่อมั่นระหว่างผู้ใช้ที่ไม่รู้จักกัน ซึ่งสอดคล้องกับการที่ระบบโครงงานนี้ออกแบบให้มีระบบรายงานประกาศและการยืนยันส่งมอบอย่างเป็นขั้นตอน

2.2 งานวิจัยเกี่ยวกับระบบแบ่งปันและซื้อขายของมือสอง

งานวิจัยด้านแพลตฟอร์มของมือสองส่วนใหญ่เน้นการซื้อขายเพื่อกำไร เช่น ระบบประมูลและระบบร้านค้าออนไลน์ ซึ่งมีกระบวนการชำระเงินและการจัดส่งเป็นศูนย์กลาง แต่สำหรับการแบ่งปันของเหลือใช้ กระบวนการหลักคือการแสดงความประสงค์ขอรับ การอนุมัติโดยเจ้าของ และการนัดรับของด้วยตนเอง ระบบจึงต้องออกแบบสถานะของรายการให้แตกต่างออกไป โครงงานนี้จึงออกแบบสถานะของการขอรับ ได้แก่ รอการพิจารณา (pending) อนุมัติ (approved) ปฏิเสธ (rejected) ยกเลิก (cancelled) และส่งมอบแล้ว (completed) เพื่อให้การติดตามรายการเป็นไปอย่างชัดเจนแต่ต้น

2.3 สถาปัตยกรรม REST API

REST (Representational State Transfer) คือรูปแบบสถาปัตยกรรมสำหรับระบบกระจายที่ Fielding (2000) เสนอขึ้น โดยใช้หลักการของ HTTP อย่างตรงไปตรงมา ทรัพยากรทุกอย่างถูกระบุด้วย URL และการทำงานกับทรัพยากรใช้วิธีการ HTTP มาตรฐาน เช่น GET สำหรับอ่าน POST สำหรับสร้าง PATCH สำหรับแก้ไข และ DELETE สำหรับลบ ข้อดีของการออกแบบระบบเป็น REST API คือส่วนหลังบ้านแยกออกจากส่วนหน้าอย่างสมบูรณ์ ทำให้สามารถเปลี่ยนเทคโนโลยีฝั่งหน้าหรือเพิ่มช่องทางใหม่ เช่น แอปพลิเคชันมือถือ โดยไม่ต้องแก้ไขส่วนหลังบ้าน อีกทั้งการจัดกลุ่ม API เป็นเวอร์ชัน เช่น /api/v1 ช่วยให้การปรับปรุงระบบในอนาคตไม่กระทบผู้ใช้เดิม

2.4 ฐานข้อมูล MongoDB และการค้นหาเชิงภูมิศาสตร์

MongoDB คือฐานข้อมูลแบบเอกสาร (Document Database) ที่จัดเก็บข้อมูลในรูปแบบ BSON ซึ่งมีความยืดหยุ่นในการออกแบบโครงสร้างข้อมูลเหมาะกับระบบที่มีข้อมูลหลากหลายรูปแบบ เช่น ประกาศที่มีรายละเอียดต่างกันตามชนิดของสินค้า จุดเด่นที่สำคัญอีกประการคือการสนับสนุนข้อมูลเชิงภูมิศาสตร์ในตัว โดย MongoDB สามารถจัดเก็บตำแหน่งในรูปแบบ GeoJSON และสร้างดัชนีชนิด 2dsphere เพื่อค้นหาเอกสารภายในระยะทางที่กำหนดได้ (MongoDB, 2025) ระบบโครงงานนี้จึงใช้ความสามารถดังกล่าวในการค้นหาประกาศใกล้ตัวผู้ใช้โดยระบุละติจูด ลองจิจูด และรัศมีค้นหาเป็นเมตร

2.5 การยืนยันตัวตนด้วย Session

การยืนยันตัวตนเป็นส่วนสำคัญของระบบที่มีการลงประกาศและอนุมัติรายการ แนวทางที่นิยมมี 2 แบบ คือ การใช้ Token เช่น JWT และการใช้ Session ฝั่งเซิร์ฟเวอร์ การใช้ Session ให้เซิร์ฟเวอร์ควบคุมสถานะการเข้าสู่ระบบได้โดยตรง สามารถยกเลิก Session ได้ทันทีเมื่อผู้ใช้ออกจากระบบหรือเมื่อบัญชีถูกระงับ และข้อมูลสำคัญไม่ถูกฝังอยู่ในตัว Token ซึ่งช่วยลดความเสี่ยงด้านความปลอดภัย ตามคำแนะนำด้านการจัดการ Session ของ OWASP (2024) ระบบโครงงานนี้จึงเลือกใช้ express-session โดยจัดเก็บ Session ลงใน MongoDB ผ่าน connect-mongo เพื่อให้ข้อมูลการเข้าสู่ระบบยังคงอยู่แม้มีการรีสตาร์ตเซิร์ฟเวอร์

2.6 เอกสาร API อัตโนมัติด้วย OpenAPI และ Swagger

OpenAPI Specification คือ มาตรฐาน สากล สำหรับ การ อธิบาย REST API ที่ ระบุ ราย ละเอียด ทรัพยากร พารามิเตอร์ และ ผลลัพธ์ ของ แต่ละ Endpoint (OpenAPI Initiative, 2024) เมื่อ นำ มา ใช้ ร่วม กับ เครื่อง มือ อย่าง Swagger UI นักพัฒนาสามารถทดลองเรียกใช้ API ได้จากหน้าเอกสารโดยตรง ทำให้การทำงานร่วมกันระหว่างผู้พัฒนาส่วนหลังบ้านและส่วนหน้าง่ายขึ้น ระบบโครงงานนี้จัดทำเอกสาร API โดยอัตโนมัติจากซอร์สโค้ดด้วย swagger-jsdoc และแสดงผลด้วย swagger-ui-express ครอบคลุม Endpoint ทั้ง 20 รายการ

2.7 ระบบเทมเพลตฝั่งเซิร์ฟเวอร์ EJS

EJS (Embedded JavaScript Templates) คือเทมเพลตเอนจินสำหรับภาษา JavaScript ที่ใช้สร้างหน้าเว็บแบบฝั่งเซิร์ฟเวอร์ (Server-Side Rendering) โดยผสานข้อมูลจากระบบหลังบ้านเข้ากับโครงสร้างหน้าเว็บ HTML ผ่านแท็กฝังโค้ดในไฟล์เทมเพลต ข้อดีของแนวทางนี้คือทำงานบนเซิร์ฟเวอร์เดียวกับ REST API โดยไม่ต้องพัฒนาส่วนหน้าแยกออกไป ผู้ใช้จึงเข้าถึงระบบได้ผ่านเบราว์เซอร์โดยตรง และการยืนยันตัวตนด้วย Session Cookie ทำงานได้ทันทีโดยไม่ต้องจัดการ Token ฝั่งลูกข่ายเพิ่มเติม อีกทั้งยังคงเส้นทาง REST API เดิมไว้สำหรับการต่อยอดสู่แอปพลิเคชันมือถือหรือเฟรมเวิร์กฝั่งหน้าในอนาคตได้อย่างเป็นอิสระต่อกัน

บทที่ 3

อุปกรณ์และวิธีดำเนินการ

การพัฒนาระบบเว็บแอปพลิเคชันแบ่งปันของเหลือใช้มีอุปกรณ์ซอฟต์แวร์ การออกแบบระบบ และวิธีการทดสอบ ดังต่อไปนี้

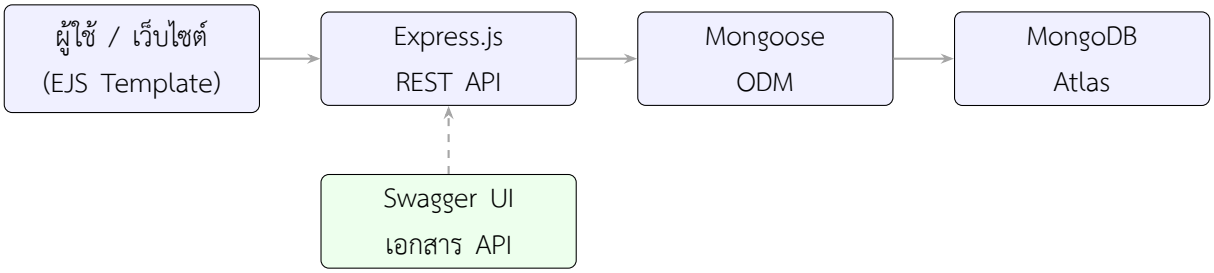
3.1 อุปกรณ์และซอฟต์แวร์

1. เครื่องคอมพิวเตอร์ ระบบปฏิบัติการ Windows
2. Node.js เวอร์ชัน 24 สำหรับรัน JavaScript ฝั่งเซิร์ฟเวอร์
3. Express.js เฟรมเวิร์กสำหรับพัฒนา REST API
4. MongoDB Atlas ฐานข้อมูลบนคลาวด์ และ Mongoose สำหรับจัดการข้อมูล
5. express-session และ connect-mongo สำหรับระบบยืนยันตัวตน
6. bcryptjs สำหรับเข้ารหัสรหัสผ่าน
7. swagger-ui-express และ swagger-jsdoc สำหรับเอกสาร API
8. EJS Template Engine สำหรับพัฒนาหน้าจอผู้ใช้ฝั่งเซิร์ฟเวอร์
9. multer สำหรับรับและจัดเก็บไฟล์รูปภาพที่ผู้ใช้อัปโหลด
10. Semantic UI สำหรับชุดองค์ประกอบหน้าจอ (UI Components) และ Font Awesome สำหรับชุดไอคอน
11. ชุดข้อมูลจังหวัด อำเภอ ตำบลของประเทศไทย จากโครงการ thailand-geography-data (เปิดเผยสาธารณะ) สำหรับ Dropdown แบบลำดับขั้น
12. Git Bash และโปรแกรม Visual Studio Code สำหรับพัฒนาโปรแกรม

3.2 การออกแบบและพัฒนาระบบ

3.2.1 สถาปัตยกรรมระบบ

ระบบออกแบบตามสถาปัตยกรรม REST API โดยแยกส่วนการทำงานอย่างชัดเจน ผู้ใช้หรือโปรแกรมฝั่งหน้าส่งคำขอผ่าน HTTP มายังเซิร์ฟเวอร์ Express.js ซึ่งตรวจสอบสิทธิ์ผู้ใช้ด้วย Session ก่อนส่งต่อให้ Controller ทำหน้าที่ประมวลผล แล้วจึงติดต่อฐานข้อมูล MongoDB ผ่าน Mongoose ดังภาพที่ 1



ภาพที่ 1: สถาปัตยกรรมของระบบ

โครงสร้างโปรเจกต์แบ่งตามหน้าที่การทำงาน ประกอบด้วยโฟลเดอร์ config สำหรับการเชื่อมต่อฐานข้อมูลและตั้งค่า Swagger, models สำหรับนิยามโครงสร้างข้อมูล, controllers และ services สำหรับตรรกะการทำงาน, middlewares สำหรับตรวจสอบสิทธิ์และจัดการข้อผิดพลาด, routes สำหรับกำหนดเส้นทาง API ทั้ง 8 กลุ่มและเส้นทางหน้าเว็บ และ views สำหรับเทมเพลตหน้าเว็บ EJS พร้อมไฟล์สไตล์ CSS ในโฟลเดอร์ public

3.2.2 การพัฒนาหน้าจอผู้ใช้ด้วย EJS

หน้าจอผู้ใช้พัฒนาด้วยเทมเพลต EJS ฝั่งเซิร์ฟเวอร์ โดยออกแบบให้ทุกหน้าใช้ส่วนหัวและส่วนท้ายร่วมกันผ่านเทมเพลตย่อย (Partial) แสดงสถานะการเข้าสู่ระบบของผู้ใช้และข้อความแจ้งเตือนชั่วคราว (Flash Message) ที่จัดเก็บใน Session หน้าเว็บหลักประกอบด้วย 1) หน้ารวมประกาศที่แสดงรายการของแบ่งปันพร้อมฟอร์มค้นหาตามหมวดหมู่และคำค้นหา 2) หน้ารายละเอียดประกาศที่แสดงข้อมูลของ ตำแหน่งรับของ และฟอร์มยื่นคำขอรับสำหรับผู้ใช้ทั่วไป หรือรายชื่อผู้ขอพร้อมปุ่มอนุมัติ ปฏิเสธ และยืนยันการส่งมอบสำหรับเจ้าของประกาศ 3) ฟอร์มสมัครสมาชิกและเข้าสู่ระบบ 4) ฟอร์มลงประกาศใหม่ และ 5) หน้ารวมประกาศของฉันและคำขอรับของฉันสำหรับติดตามสถานะรายการ

การทำงานของหน้าเว็บยึดหลักการสำคัญคือใช้ Session เดียวกันกับ REST API ทำให้ผู้ใช้ที่เข้าสู่ระบบผ่านหน้าเว็บสามารถเรียกใช้ API ได้ทันที และการกระทำทุกอย่างบนหน้าเว็บผ่านการตรวจสอบสิทธิ์ชุดเดียวกัน เมื่อดำเนินการสำเร็จระบบจะเปลี่ยนหน้า (Redirect) พร้อมข้อความยืนยัน หากเกิดข้อผิดพลาด เช่น จองซ้ำหรือของไม่พอ ระบบจะแสดงข้อความแจ้งเตือนกลับมาที่หน้าเดิมแทนการแสดงรหัสข้อผิดพลาดดิบ ๆ

3.2.3 การออกแบบฐานข้อมูล

ระบบออกแบบคอลเลกชันหลัก 5 คอลเลกชัน ดังแสดงในตารางที่ 1 โดยความสัมพันธ์หลักคือผู้ใช้หนึ่งคนสามารถมีประกาศได้หลายประกาศ และประกาศหนึ่งประกาศสามารถมีคำขอรับได้หลายคำขอ โดยแยกคำขอรับเป็นคอลเลกชันต่างหากเพื่อให้จัดการสถานะได้ง่ายและป้องกันการจองซ้ำด้วยดัชนีเอกลักษณ์

ตารางที่ 1: คอลเลกชันหลักของระบบและหน้าที่

คอลเลกชัน	หน้าที่และข้อมูลสำคัญ
User	ข้อมูลผู้ใช้ ชื่อ อีเมล รหัสผ่านแบบเข้ารหัส บทบาท (ผู้ใช้ทั่วไปหรือผู้ดูแลระบบ) และที่อยู่
Category	หมวดหมู่ของแบ่งปัน 7 หมวด เช่น อาหาร เสื้อผ้า เฟอร์นิเจอร์ เครื่องใช้ไฟฟ้า
Listing	ประกาศของแบ่งปัน ชื่อ รายละเอียด สภาพของ จำนวนและจำนวนคงเหลือ สถานะ วิธีรับของ และตำแหน่งพิกัด
Reservation	คำขอรับของ ผู้ขอ จำนวนที่ขอ ข้อความ และสถานะ ตั้งแต่รอพิจารณาจนถึงส่งมอบ
Report	การรายงานประกาศไม่เหมาะสม เหตุผล และสถานะการตรวจสอบ

นอกจากนี้ระบบยังสร้างดัชนีสำคัญไว้ล่วงหน้า เช่น ดัชนีอีเมลแบบเอกลักษณ์ของผู้ใช้ ดัชนีค้นหาแบบเต็มข้อความบนชื่อและรายละเอียดประกาศ ดัชนี 2dsphere บนตำแหน่งรับของเพื่อการค้นหาตามระยะทาง และดัชนีเอกลักษณ์คู่ (ประกาศ, ผู้ขอ) บนคำขอรับเพื่อป้องกันการจองซ้ำจากผู้ใช้เดียวกัน

3.2.4 พจนานุกรมข้อมูล (Data Dictionary)

ตารางที่ 2 ถึงตารางที่ 6 แสดงรายละเอียดฟิลด์ข้อมูลของแต่ละคอลเลกชันทั้ง 5 คอลเลกชันที่ออกแบบด้วย Mongoose Schema

3.2.5 โครงสร้างข้อมูล JSON (JSON Map Structure)

เนื่องจาก MongoDB จัดเก็บข้อมูลในรูปแบบ BSON ซึ่งมีโครงสร้างตรงกับ JSON และ REST API ของระบบรับส่งข้อมูลทั้งหมดเป็น JSON จากพจนานุกรมข้อมูลในหัวข้อที่ผ่านมา จึงสามารถสรุปโครงสร้าง JSON ของเอกสารหลักที่ระบบใช้งานจริงได้ดังตัวอย่างต่อไปนี้

ตัวอย่างเอกสาร JSON ของคอลเลกชัน Listing (ประกาศของแบ่งปัน):

```
{
  "_id": "6700a1b2c3d4e5f6a7b8c9d0",
  "owner": "66ff...u1",
  "category": "66ff...c3",
  "title": "Wooden Chair - Good Condition",
  "description": "Solid wood chair, still fully usable, free to
      a good home",
  "images": ["/assets/uploads/1699999999-chair.jpg"],
  "condition": "good",
  "quantity": 1,
  "availableQuantity": 1,
```

ตารางที่ 2: พจนานุกรมข้อมูลคอลเลกชัน User

ฟิลด์	ชนิดข้อมูล	ข้อกำหนด	คำอธิบาย
_id	ObjectId	Primary Key	รหัสผู้ใช้ (สร้างอัตโนมัติ)
name	String	จำเป็น	ชื่อ-นามสกุล
email	String	จำเป็น, ไม่ซ้ำ	อีเมลเข้าสู่ระบบ (unique)
passwordHash	String	จำเป็น	รหัสผ่านเข้ารหัสด้วย bcrypt
phone	String	ไม่บังคับ	เบอร์โทรศัพท์
avatarUrl	String	ไม่บังคับ	ลิงก์รูปโปรไฟล์
role	String (enum)	default: user	บทบาท: user หรือ admin
location	GeoJSON Point	ไม่บังคับ	พิกัดผู้ใช้ (ดัชนี 2dsphere)
address.province/ district/subdistrict	String	ไม่บังคับ	ที่อยู่ จังหวัด/อำเภอ/ตำบล
isActive	Boolean	default: true	สถานะบัญชี (false = ถูกระงับ)
banReason	String	ไม่บังคับ	เหตุผลที่ถูกระงับ
bannedAt	Date	ไม่บังคับ	วันที่ถูกระงับ
createdAt / updatedAt	Date	อัตโนมัติ	วันที่สร้าง/แก้ไข

ตารางที่ 3: พจนานุกรมข้อมูลคอลเลกชัน Category

ฟิลด์	ชนิดข้อมูล	ข้อกำหนด	คำอธิบาย
_id	ObjectId	Primary Key	รหัสหมวดหมู่
name	String	จำเป็น	ชื่อหมวดหมู่
slug	String	จำเป็น, ไม่ซ้ำ	ชื่อย่ออ้างอิงใน URL
description	String	ไม่บังคับ	คำอธิบายหมวดหมู่
icon	String	ไม่บังคับ	ไอคอนหมวดหมู่ (emoji)
isActive	Boolean	default: true	เปิด/ปิดใช้งาน (Soft Delete)
createdAt / updatedAt	Date	อัตโนมัติ	วันที่สร้าง/แก้ไข

ตารางที่ 4: พจนานุกรมข้อมูลคอลเลกชัน Listing

ฟิลด์	ชนิดข้อมูล	ข้อกำหนด	คำอธิบาย
_id	ObjectId	Primary Key	รหัสประกาศ
owner	ObjectId (ref: User)	จำเป็น	ผู้ลงประกาศ
category	ObjectId (ref: Category)	จำเป็น	หมวดหมู่สิ่งของ
title	String	จำเป็น	ชื่อสิ่งของ
description	String	จำเป็น	รายละเอียดสิ่งของ
images	[String]	1–5 รายการ	ลิงก์รูปภาพ (URL/อัปโหลด)
condition	String (enum)	จำเป็น	สภาพ: new, like_new, good, fair, poor
quantity	Number	default: 1, min: 1	จำนวนทั้งหมด
availableQuantity	Number	default: 1, min: 0	จำนวนคงเหลือ
status	String (enum)	default: available	available, reserved, completed, cancelled, expired
pickup.method	String (enum)	fixed: pickup	วิธีรับของ (pickup เท่านั้น)
pickup.location	GeoJSON Point	ไม่บังคับ	พิกัดนัดรับ (ปกปิดจนอนุมัติ)
pickup.address	String	ไม่บังคับ	ที่อยู่นัดรับ (ปกปิดจนอนุมัติ)
pickup.area. province/district/ subdistrict	String	จำเป็น	พื้นที่โดยประมาณ (สาธารณะ)
expiresAt	Date	ไม่บังคับ	วันหมดอายุประกาศ
viewCount	Number	default: 0	จำนวนครั้งที่เปิดดู
createdAt / updatedAt	Date	อัตโนมัติ	วันที่สร้าง/แก้ไข

ตารางที่ 5: พจนานุกรมข้อมูลคอลเลกชัน Reservation

ฟิลด์	ชนิดข้อมูล	ข้อกำหนด	คำอธิบาย
_id	ObjectId	Primary Key	รหัสคำขอรับ
listing	ObjectId (ref: Listing)	จำเป็น	ประกาศที่ขอรับ
requester	ObjectId (ref: User)	จำเป็น	ผู้ยื่นคำขอ
quantity	Number	default: 1, min: 1	จำนวนที่ขอรับ
message	String	ไม่บังคับ	ข้อความถึงเจ้าของ
status	String (enum)	default: pending	pending, approved, rejected, cancelled, completed
approvedAt / completedAt / cancelledAt	Date	ไม่บังคับ	วันที่เปลี่ยนสถานะ
isReadByRequester	Boolean	default: true	สถานะอ่าน (ใช้แสดง Badge)
createdAt / updatedAt	Date	อัตโนมัติ	วันที่สร้าง/แก้ไข

หมายเหตุ: มีดัชนีเอกลักษณ์คู่ (listing, requester) เพื่อบังคับว่าผู้ใช้หนึ่งคนยื่นคำขอต่อประกาศหนึ่งประกาศได้เพียงครั้งเดียวตลอดไป (Single Lifetime Reservation)

ตารางที่ 6: พจนานุกรมข้อมูลคอลเลกชัน Report

ฟิลด์	ชนิดข้อมูล	ข้อกำหนด	คำอธิบาย
_id	ObjectId	Primary Key	รหัสรายงาน
reporter	ObjectId (ref: User)	จำเป็น	ผู้แจ้งรายงาน
listing	ObjectId (ref: Listing)	จำเป็น	ประกาศที่ถูกรายงาน
reason	String (enum)	จำเป็น	spam, inappropriate, fraud, illegal, wrong_information, other
description	String	ไม่บังคับ	รายละเอียดเพิ่มเติม
status	String (enum)	default: pending	pending, reviewing, resolved, rejected
createdAt / updatedAt	Date	อัตโนมัติ	วันที่สร้าง/แก้ไข

```
  "status": "available",
  "pickup": {
    "method": "pickup",
    "area": { "province": "Khon Kaen", "district": "Mueang Khon
        Kaen", "subdistrict": "Nai Mueang" },
    "location": { "type": "Point", "coordinates": [102.8333,
       16.4322] }
  },
  "expiresAt": null,
  "viewCount": 12,
  "createdAt": "2026-09-01T03:12:00.000Z",
  "updatedAt": "2026-09-01T03:12:00.000Z"
}
```

ตัวอย่างเอกสาร JSON ของคอลเลกชัน Reservation (คำขอรับของ):

```
{
  "_id": "6700b2c3d4e5f6a7b8c9d0e1",
  "listing": "6700a1b2c3d4e5f6a7b8c9d0",
  "requester": "66ff...u2",
```

```
  "quantity": 1,
  "message": "Interested in the chair, can I pick it up on
     Saturday?",
  "status": "pending",
  "isReadByRequester": true,
  "createdAt": "2026-09-02T08:40:00.000Z",
  "updatedAt": "2026-09-02T08:40:00.000Z"
}
```

จะเห็นได้ว่าความสัมพันธ์ระหว่างคอลเลกชันอ้างอิงกันด้วย **ObjectId** (เช่น **listing** และ **requester** ใน Reservation อ้างอิงกลับไปยัง **_id** ของ Listing และ User ตามลำดับ) ซึ่งเป็นรูปแบบเดียวกับที่แสดงในตารางที่ 2 ถึงตารางที่ 6

3.2.6 โครงสร้าง API (API Map Structure)

ระบบออกแบบเส้นทาง REST API ทั้งหมดให้ขึ้นต้นด้วย **/api/v1** และแบ่งกลุ่มตามทรัพยากร (Resource-based Routing) รวม 8 กลุ่ม ดังแผนผังต่อไปนี้

```
/api/v1
|-- /auth           (register, login, logout, me, change
   password)
|-- /users           (profile, my listings, manage users --
   admin only)
|-- /categories       (list / create / update / disable
   categories)
|-- /listings         (create, search, update, cancel a listing
   )
|-- /reservations      (request, approve, reject, cancel,
   complete, unread)
|-- /reports          (report a listing, review by admin)
|-- /admin           (system overview stats -- admin only)
`-- /uploads          (upload an image file)
```

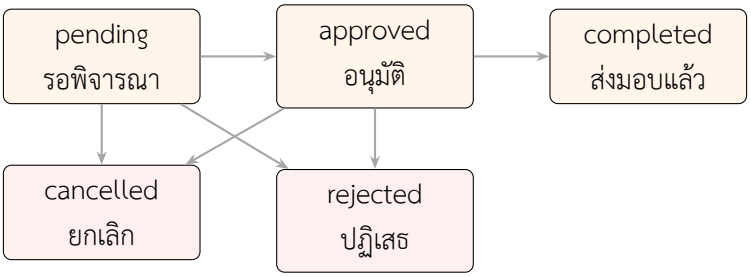
แต่ละกลุ่มยึดรูปแบบ HTTP Method ตามหลัก REST คือ **GET** สำหรับอ่านข้อมูล **POST** สำหรับสร้างข้อมูลใหม่ **PATCH** สำหรับแก้ไขบางส่วน และ **DELETE**/การเปลี่ยนสถานะสำหรับยกเลิกข้อมูล รวมทั้งสิ้น 38 Endpoint ตามที่สรุปไว้ในตารางที่ 8 พร้อมตัวอย่างคำขอ/ผลลัพธ์ในภาคผนวก A และรายละเอียดครบทุก Endpoint ในภาคผนวก C

3.2.7 กระบวนการขอรับและส่งมอบของ

เมื่อผู้ใช้พบประกาศที่ต้องการจะยื่นคำขอรับพร้อมระบุจำนวนและข้อความ ระบบจะตรวจสอบว่าประกาศพร้อมรับคำขอ ผู้ขอไม่ใช่เจ้าของประกาศเอง และจำนวนที่ขอไม่เกินจำนวนคงเหลือ เจ้าของประกาศสามารถอนุมัติคำขอได้ โดยระบบจะลดจำนวนคงเหลือแบบอะตอมมิกเพื่อป้องกันการอนุมัติเกินจำนวนเมื่อมีหลายคำขอพร้อมกัน ตามเงื่อนไข

$$A' = A - q \quad \text{เมื่อ} \quad A \geq q$$

เมื่อ A คือจำนวนคงเหลือ และ q คือจำนวนที่ขออนุมัติ หากจำนวนคงเหลือเป็นศูนย์ ระบบจะเปลี่ยนสถานะประกาศเป็น reserved และเมื่อเจ้าของยืนยันว่าส่งมอบของแล้ว คำขอจะเปลี่ยนเป็น completed และประกาศจะถูกปิดเป็น completed ดังภาพที่ 2



ภาพที่ 2: สถานะของคำขอรับของ (Reservation)

3.3 ข้อมูลจำเพาะของระบบที่ใช้ทดสอบและนำเสนอ (System Specifications)

ตารางที่ 7 แสดงข้อมูลจำเพาะของเครื่องและซอฟต์แวร์ที่คณะผู้จัดทำใช้ในการพัฒนา ทดสอบ และนำเสนอระบบจริง

3.4 การทดสอบระบบ

การทดสอบใช้วิธีทดสอบการทำงานจริง (Smoke Test) โดยเรียกใช้ API ผ่านเครื่องมือ curl ตามลำดับสถานการณ์การใช้งานจริง ตั้งแต่การสมัครสมาชิกจนถึงการปิดรายการ โดยกำหนดเกณฑ์การผ่านจากค่ารหัสสถานะ HTTP และค่าข้อมูลที่ระบบตอบกลับ เช่น การสมัครสำเร็จต้องได้รหัส 201 การเข้าถึงโดยไม่ยืนยันตัวตนต้องได้รหัส 401 และการจองซ้ำต้องได้รหัส 409 ทั้งนี้ทดสอบครอบคลุม 10 สถานการณ์ดังแสดงในบทที่ 4

ตารางที่ 7: ข้อมูลจำเพาะของระบบที่ใช้ทดสอบและนำเสนอ

รายการ	**รายละเอียด**
ระบบปฏิบัติการ	Windows 11
Runtime ฝั่งเซิร์ฟเวอร์	Node.js เวอร์ชัน 24
เฟรมเวิร์กฝั่งเซิร์ฟเวอร์	Express.js เวอร์ชัน 5.2
ตัวจัดการฐานข้อมูล	Mongoose เวอร์ชัน 9.11 เชื่อมต่อกับ MongoDB Atlas (คลาวด์)
เบราว์เซอร์ที่ใช้ทดสอบ	Google Chrome เวอร์ชันล่าสุด
พอร์ตที่ให้บริการ	`http://localhost:3000` (REST API และหน้าเว็บ) และ `http://localhost:3000/api-docs` (เอกสาร Swagger)
เครื่องมือทดสอบ API	curl และ Swagger UI
เครื่องมือพัฒนา	Visual Studio Code และ Git Bash

บทที่ 4

ผลการดำเนินงาน

จากการพัฒนาตามที่ออกแบบไว้ ระบบสามารถทำงานได้ครบทุกส่วน ผลการดำเนินงานสรุปได้ดังนี้

4.1 ผลการพัฒนาระบบ

ระบบพัฒนาเสร็จสมบูรณ์โดยแบ่ง API ออกเป็น 8 กลุ่ม รวม 38 Endpoint ดังแสดงในตารางที่ 8 พร้อมเอกสารอัตโนมัติที่เข้าใช้งานได้ผ่านหน้า Swagger UI และสามารถเชื่อมต่อกับฐานข้อมูล MongoDB Atlas บนคลาวด์ได้จริง นอกจากนี้ยังพัฒนาหน้าจอผู้ใช้ด้วยเทมเพลต EJS ครบทั้งหมด 19 เส้นทางหน้าเว็บ ครอบคลุมฝั่งผู้ใช้ทั่วไป ฝั่งเจ้าของประกาศ และฝั่งผู้ดูแลระบบ (Admin Back-office) โดยเส้นทางหน้าเว็บทั้งหมดแสดงรายการในภาคผนวก C

ตารางที่ 8: กลุ่ม API และหน้าที่หลักของแต่ละกลุ่ม

กลุ่ม	จำนวน	หน้าที่หลัก
Auth	5	สมัครสมาชิก เข้าสู่ระบบ ดูข้อมูลตนเอง ออกจากระบบ เปลี่ยนรหัสผ่าน
Users	7	จัดการข้อมูลส่วนตัว ดูประกาศของตนเอง และจัดการผู้ใช้สำหรับผู้ดูแลระบบ
Categories	5	จัดการหมวดหมู่ของแบ่งปัน แก้ไขได้โดยผู้ดูแลระบบ
Listings	5	ลงประกาศ ค้นหา แก้ไข และยกเลิกประกาศ (Soft Delete)
Reservations	10	ยื่นคำขอรับ อนุมัติ ปฏิเสธ ยกเลิก ยืนยันการส่งมอบ และแจ้งเตือนที่ยังไม่อ่าน
Reports	4	รายงานประกาศไม่เหมาะสม และตรวจสอบโดยผู้ดูแลระบบ
Admin	1	สรุปตัวเลขภาพรวมระบบสำหรับหน้า Admin Overview
Uploads	1	รับและจัดเก็บไฟล์รูปภาพที่ผู้ใช้อัปโหลด
รวม	**38**	

4.2 การตรวจพบข้อบกพร่องและการพัฒนาเพิ่มเติม

ก่อนเริ่มพัฒนาต่อในช่วงท้ายของโครงงาน คณะผู้จัดทำได้ทบทวนระบบที่พัฒนาไว้เดิมอย่างละเอียด โดยเปรียบเทียบพฤติกรรมจริงของโค้ดกับกฎทางธุรกิจที่ออกแบบไว้ในเอกสารข้อกำหนด พบข้อบกพร่องสำคัญหลายจุดทั้งด้านความปลอดภัยและตรรกะการทำงาน รวมถึงพบว่าหน้าจอผู้ใช้ (Web UI) ยังพัฒนาไม่ครบตามที่ออกแบบไว้ จึงดำเนินการแก้ไขและพัฒนาเพิ่มเติมจนครบถ้วน สรุปได้ดังตารางที่ 9

ตารางที่ 9: สรุปข้อบกพร่องที่ตรวจพบและการแก้ไข/พัฒนาเพิ่มเติม

ด้าน	สภาพก่อนปรับปรุง	การแก้ไข/พัฒนาเพิ่มเติม
ความปลอดภัย: สิทธิ์ผู้ใช้	`PATCH /users/:id/status` ยอมรับฟิลด์ `role` ทำให้สามารถเลื่อนสิทธิ์ผู้ใช้เป็นผู้ดูแลระบบผ่าน API ได้โดยตรง	แก้ไขให้ Endpoint นี้รับเฉพาะ `isActive` และ `reason` เท่านั้น การกำหนดบทบาทผู้ดูแลระบบทำได้เฉพาะระดับฐานข้อมูลโดยตรง
ความปลอดภัย: บัญชีถูกระงับ	Middleware ตรวจสอบสิทธิ์ (`requireAuth`) อ้างอิงข้อมูลจาก Session เท่านั้น ผู้ใช้ที่ถูกระงับการใช้งานแล้วยังคงใช้งานต่อได้จนกว่า Session จะหมดอายุ	ปรับให้ตรวจสอบสถานะ `isActive` จากฐานข้อมูลสดใหม่ทุกคำขอ และตอบกลับรหัส 401 ทันทีเมื่อบัญชีถูกระงับ
ความเป็นส่วนตัวของข้อมูล	`GET /listings/:id` ส่งเบอร์โทรเจ้าของประกาศ ที่อยู่ละเอียด และพิกัดที่ตั้งจริงให้ผู้เรียกทุกคนโดยไม่มีการปกปิด	เพิ่มกลไก Privacy Masking: แสดงข้อมูลติดต่อและที่อยู่ละเอียดเฉพาะเจ้าของประกาศหรือผู้ขอที่ได้รับอนุมัติแล้วเท่านั้น ผู้อื่นเห็นเพียงพื้นที่โดยประมาณ
สิทธิ์ของผู้ดูแลระบบ	ผู้ดูแลระบบสามารถแก้ไขเนื้อหาประกาศหรืออนุมัติ/ปฏิเสธคำขอแทนเจ้าของประกาศได้	จำกัดสิทธิ์ผู้ดูแลระบบให้ทำได้เฉพาะการยกเลิกประกาศ (ตรวจสอบ/กำกับดูแล) ไม่สามารถแก้ไขเนื้อหาหรือกระทำการแทนเจ้าของได้
การแก้ไขประกาศ	ประกาศที่มีผู้ยื่นคำขอรับแล้วยังสามารถแก้ไขได้อยู่ (Strict Edit Lock ยังไม่ถูกบังคับใช้จริง)	เพิ่มการตรวจสอบและบังคับห้ามแก้ไขทันทีที่มีคำขอรับเกิดขึ้นแล้ว ไม่ว่าจะอยู่ในสถานะใด
การยกเลิกประกาศ	การยกเลิกประกาศเปลี่ยนเฉพาะสถานะประกาศ ไม่ยกเลิกคำขอรับที่ค้างอยู่หรือคืนจำนวนคงเหลือ	เพิ่มกลไก Cascade Cancellation: คำขอที่รอพิจารณาและที่อนุมัติแล้วถูกยกเลิกอัตโนมัติ พร้อมคืนจำนวนคงเหลือให้ถูกต้อง
การปิดรายการเมื่อของมีหลายชิ้น	ระบบปิดประกาศเป็น completed ทันทีที่จำนวนคงเหลือเป็นศูนย์ ซึ่งคลาดเคลื่อนเมื่อยังส่งมอบของไม่ครบทุกคำขอ	แก้สูตรให้ปิดประกาศเฉพาะเมื่อผลรวมจำนวนที่ส่งมอบสำเร็จเท่ากับจำนวนทั้งหมดของประกาศเท่านั้น

(ต่อจากหน้าก่อน) ตารางที่ 9: สรุปข้อบกพร่องที่ตรวจพบและการแก้ไข/พัฒนาเพิ่มเติม

ด้าน	สภาพก่อนปรับปรุง	การแก้ไข/พัฒนาเพิ่มเติม
การแจ้งเตือน	ยังไม่มีกลไกแจ้งเตือนเมื่อสถานะคำขอเปลี่ยนแปลง	เพิ่มฟิลด์ `isReadByRequester` และ Endpoint กลุ่มใหม่ (unread-count, read, read-all) พร้อม Badge แจ้งเตือนบนหน้าเว็บ
หน้าจอผู้ใช้ (Web UI)	มีเพียงหน้าตัวอย่างสาธิตองค์ประกอบ (Phase 0) ยังไม่มีหน้าใช้งานจริงแม้แต่หน้าเดียว	พัฒนาหน้าเว็บครบทั้ง 19 เส้นทาง ครอบคลุมผู้เยี่ยมชม สมาชิก เจ้าของประกาศ และผู้ดูแลระบบ (รายละเอียดในภาคผนวก C)
ระบบหลังบ้านผู้ดูแลระบบ	ยังไม่มีหน้าจอสำหรับผู้ดูแลระบบ	พัฒนาเพิ่มใหม่ 5 หน้า ได้แก่ ภาพรวมระบบ จัดการประกาศ จัดการรายงาน จัดการผู้ใช้ และจัดการหมวดหมู่
การอัปโหลดรูปภาพ	รองรับเฉพาะการระบุลิงก์ URL ภายนอกเท่านั้น	เพิ่ม Endpoint `POST /uploads` สำหรับอัปโหลดไฟล์จากเครื่องโดยตรง ใช้ร่วมกับการระบุ URL ภายนอกได้
การกรอกที่อยู่	กรอกจังหวัด/อำเภอ/ตำบลด้วยข้อความอิสระ สะกดไม่ตรงกันได้	เปลี่ยนเป็น Dropdown แบบลำดับขั้น (จังหวัด ⟶ อำเภอ ⟶ ตำบล) อ้างอิงข้อมูลภูมิศาสตร์จริงของประเทศไทย
รูปลักษณ์หน้าจอ	ใช้ชุดสไตล์ CSS และสัญลักษณ์ Emoji ที่พัฒนาขึ้นเอง	ออกแบบใหม่ทั้งระบบด้วยไลบรารี Semantic UI ไอคอน Font Awesome และฟอนต์ Kanit เพื่อความสม่ำเสมอของส่วนติดต่อผู้ใช้

ข้อบกพร่องด้านความปลอดภัยและตรรกะทางธุรกิจข้างต้นถูกตรวจสอบยืนยันซ้ำด้วยการทดสอบจริงผ่านเบราว์เซอร์และการเรียก API โดยตรงหลังการแก้ไขทุกจุด เพื่อยืนยันว่าพฤติกรรมของระบบเป็นไปตามกฎที่ออกแบบไว้แล้วจริง

4.3 ผลการทดสอบระบบ

ผลการทดสอบทั้ง 10 สถานการณ์แสดงในตารางที่ 10 พบว่าทุกสถานการณ์ผ่านตามเกณฑ์ที่กำหนดไว้

จากตารางที่ 10 จะเห็นว่าระบบทำงานได้ตามที่ออกแบบไว้ทุกสถานการณ์ โดยเฉพาะการป้องกันการจองซ้ำด้วยดัชนีเอกลักษณ์ซึ่งระบบปฏิเสธคำขอที่ซ้ำอัตโนมัติ และการอนุมัติที่ลดจำนวนคงเหลือแบบอะตอม

ตารางที่ 10: ผลการทดสอบระบบในแต่ละสถานการณ์

ลำดับ	สถานการณ์ทดสอบ	ผลที่คาดหวัง	ผลจริง
1	สมัครสมาชิกและเข้าสู่ระบบ	201 / 200	ผ่าน
2	เข้าถึงข้อมูลโดยไม่ยืนยันตัวตน	401	ผ่าน
3	ลงประกาศพร้อมตำแหน่งพิกัด	201	ผ่าน
4	ค้นหาประกาศตามหมวดหมู่และระยะทาง	พบประกาศ	ผ่าน
5	ยื่นคำขอรับของ	201	ผ่าน
6	ผู้ใช้เดียวกันยื่นคำขอซ้ำ	409	ผ่าน
7	อนุมัติคำขอ ลดจำนวนคงเหลือ	จำนวนลดลง	ผ่าน
8	ปฏิเสธคำขอที่อนุมัติแล้ว คืนจำนวน	จำนวนคืน	ผ่าน
9	ยืนยันส่งมอบและปิดรายการ	completed	ผ่าน
10	ผู้ใช้ทั่วไปเรียกใช้งานส่วนผู้ดูแลระบบ	403	ผ่าน

มิก ซึ่งป้องกันปัญหาการอนุมัติเกินจำนวนของที่มีอยู่แม้มีหลายคำขอเกิดขึ้นพร้อมกัน นอกจากนี้การค้นหาตามระยะทางสามารถค้นพบประกาศภายในรัศมีที่กำหนดได้ถูกต้อง

สำหรับหน้าจอผู้ใช้ EJS ได้ทดสอบการแสดงผลของทุกหน้า การเข้าสู่ระบบและสมัครสมาชิกผ่านฟอร์ม การลงประกาศและยื่นคำขอรับผ่านหน้าเว็บ ตลอดจนการแสดงข้อความแจ้งเตือนเมื่อเกิดข้อผิดพลาด พบว่าหน้าเว็บทำงานได้ถูกต้องเชื่อมโยงกับส่วนหลังบ้านอย่างสมบูรณ์ และสิทธิ์การเข้าถึงแต่ละหน้าสอดคล้องกับสถานะการเข้าสู่ระบบของผู้ใช้

บทที่ 5

สรุปผลการดำเนินงาน อภิปรายผล และข้อเสนอแนะ

การศึกษาครั้งนี้เป็นการพัฒนาระบบเว็บแอปพลิเคชันแบ่งปันของเหลือใช้ด้วยสถาปัตยกรรม REST API ร่วมกับฐานข้อมูล MongoDB ซึ่งสามารถสรุปผล อภิปรายผล และข้อเสนอแนะ ดังนี้

5.1 สรุปผลการดำเนินงาน

คณะผู้จัดทำสามารถพัฒนาระบบครบตามวัตถุประสงค์ทั้ง 3 ข้อ ได้แก่ 1) ระบบลงประกาศแบ่งปันของเหลือใช้ที่ผู้ใช้สามารถระบุรายละเอียด สภาพของ จำนวน วิธีรับของ และตำแหน่งพิกัดได้ 2) ระบบขอรับ จอง อนุมัติ และปิดรายการที่มีสถานะชัดเจน พร้อมกลไกป้องกันการจองซ้ำและการอนุมัติเกินจำนวน และ 3) REST API จำนวน 38 Endpoint ใน 8 กลุ่ม พร้อมเอกสารอัตโนมัติด้วย Swagger และหน้าจอผู้ใช้ครบ 19 เส้นทาง รวมถึงระบบหลังบ้านสำหรับผู้ดูแลระบบ (Admin Back-office) ผลการทดสอบทั้ง 10 สถานการณ์ผ่านตามเกณฑ์ที่กำหนด

5.2 อภิปรายผล

จากผลการพัฒนาและทดสอบ พบว่าการออกแบบระบบโดยแยกคำขอรับออกเป็นคอลเลกชันต่างหากช่วยให้การจัดการสถานะของรายการง่ายขึ้นอย่างชัดเจน สามารถเพิ่มสถานะใหม่หรือปรับกติกาการเปลี่ยนสถานะได้โดยไม่กระทบข้อมูลประกาศ สอดคล้องกับหลักการออกแบบฐานข้อมูลเชิงสัมพันธ์ที่ควรแยกข้อมูลตามหน้าที่การทำงาน อีกทั้งการใช้ดัชนีเอกลักษณ์ของฐานข้อมูลเพื่อป้องกันการจองซ้ำมีความน่าเชื่อถือมากกว่าการตรวจสอบด้วยโค้ดเพียงอย่างเดียว เนื่องจากฐานข้อมูลรับประกันความถูกต้องในระดับพื้นที่จัดเก็บข้อมูล

การลดจำนวนคงเหลือแบบอะตอมมิกด้วยเงื่อนไขในคำสั่งอัปเดตเดียว ช่วยแก้ปัญหาการแข่งกันอนุมัติ (Race Condition) ที่อาจเกิดขึ้นเมื่อมีหลายคำขอถูกอนุมัติพร้อมกัน ทำให้จำนวนของคงเหลือถูกต้องเสมอ ซึ่งเป็นประเด็นสำคัญของระบบแบ่งปันทรัพยากรที่มีผู้ใช้หลายคนเข้าใช้พร้อมกัน และการเลือกใช้ Session แทน Token ทำให้การควบคุมการเข้าสู่ระบบ เช่น การระงับบัญชีหรือการบังคับออกจากระบบ ทำได้ทันทีจากฝั่งเซิร์ฟเวอร์

อย่างไรก็ตาม ระบบในขั้นตอนนี้ยังมีข้อจำกัดด้านการแจ้งเตือนผู้ใช้ผ่านช่องทางภายนอกระบบ เช่น อีเมลหรือแอปพลิเคชันส่งข้อความ ซึ่งเป็นสิ่งที่มีผลต่อประสบการณ์การใช้งานจริง จำเป็นต้องพัฒนาต่อเพื่อให้ระบบใช้งานได้จริงในระดับชุมชน

5.3 ข้อเสนอแนะสำหรับการวิจัยครั้งต่อไป

5.3.1 ข้อเสนอแนะสำหรับการนำผลงานไปใช้

ผู้ที่ประสงค์จะนำระบบนี้ไปพัฒนาต่อ หน้าจอผู้ใช้ที่พัฒนาด้วย EJS สามารถนำไปปรับปรุงรูปลักษณ์และประสบการณ์การใช้งานต่อได้ทันที หรือหากต้องการยกระดับสู่แอปพลิเคชันมือถือหรือเฟรมเวิร์กฝั่งหน้าแบบ Single Page Application ก็สามารถเชื่อมต่อกับ REST API ที่พัฒนาไว้ได้โดยตรงโดยอ้างอิงจากเอกสาร Swagger รวมถึงสามารถเพิ่มเติมระบบจัดการข้อความระหว่างผู้ใช้ และระบบประเมินความน่าเชื่อถือของผู้ใช้เพื่อเสริมสร้างความเชื่อมั่นในชุมชนผู้ใช้ให้สูงขึ้น

5.3.2 ข้อเสนอแนะในการศึกษาครั้งถัดไป

ในการศึกษาครั้งถัดไปควรพัฒนาระบบแจ้งเตือนเมื่อมีคำขอรับใหม่ผ่านช่องทางภายนอก เช่น อีเมลหรือแอปพลิเคชันส่งข้อความ เนื่องจากปัจจุบันระบบแจ้งเตือนทำงานเฉพาะภายในแอปพลิเคชันเท่านั้น ควรประเมินความพึงพอใจของผู้ใช้จริงด้วยแบบประเมินมาตรฐาน และทดสอบประสิทธิภาพเมื่อมีผู้ใช้จำนวนมากพร้อมกัน อีกทั้งควรศึกษาการยืนยันตัวตนแบบสองขั้น (Two-Factor Authentication) เพื่อเพิ่มความปลอดภัยของบัญชีผู้ใช้ และอาจพิจารณาจำกัดขนาด/ชนิดไฟล์รูปภาพที่อัปโหลดให้รัดกุมยิ่งขึ้นเมื่อเปิดให้ใช้งานจริงในวงกว้าง

บรรณานุกรม

[1] Botsman, R., & Rogers, R. (2010). *What's Mine Is Yours: The Rise of Collaborative Consumption*. New York: HarperBusiness.

[2] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* [Doctoral dissertation, University of California, Irvine]. ค้น เมื่อ 6 ตุลาคม 2568, จาก https://ics.uci.edu/~fielding/pubs/dissertation/fielding_dissertation.pdf

[3] Hamari, J., Sjöklint, M., & Ukkonen, A. (2016). The sharing economy: Why people participate in collaborative consumption. *Journal of the Association for Information Science and Technology*, 67(9), 2047–2059.

[4] Express. (2025). *Express – Node.js web application framework*. ค้นเมื่อ 6 ตุลาคม 2568, จาก https://expressjs.com/

[5] MongoDB. (2025). *MongoDB Manual: Geospatial Queries*. ค้นเมื่อ 6 ตุลาคม 2568, จาก https://www.mongodb.com/docs/manual/geospatial-queries/

[6] Mongoose. (2025). *Mongoose ODM Documentation*. ค้นเมื่อ 6 ตุลาคม 2568, จาก https://mongoosejs.com/docs/

[7] OpenJS Foundation. (2025). *express-session*. ค้นเมื่อ 6 ตุลาคม 2568, จาก https://www.npmjs.com/package/express-session

[8] OWASP Foundation. (2024). *Session Management Cheat Sheet*. ค้น เมื่อ 6 ตุลาคม 2568, จาก https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

[9] OpenAPI Initiative. (2024). *OpenAPI Specification 3.0.3*. ค้นเมื่อ 6 ตุลาคม 2568, จาก https://spec.openapis.org/oas/v3.0.3

[10] SmartBear Software. (2025). *swagger-ui-express*. ค้น เมื่อ 6 ตุลาคม 2568, จาก https://www.npmjs.com/package/swagger-ui-express

ภาคผนวก A

ตัวอย่างการเรียกใช้ API

ตัวอย่างต่อไปนี้เป็นการเรียกใช้ API หลักของระบบ โดยทำงานบนเซิร์ฟเวอร์ที่ทำงานอยู่ที่ที่อยู่ http://localhost:3000

1.1 สมัครสมาชิก

```
POST /api/v1/auth/register
Content-Type: application/json

{
  "name": "Somchai",
  "email": "somchai@example.com",
  "password": "password123",
  "phone": "0812345678"
}

Response: 201 Created
{
  "success": true,
  "data": { "user": { "id": "...", "name": "Somchai" } }
}
```

1.2 ลงประกาศของแบ่งปัน

```
POST /api/v1/listings
Content-Type: application/json

{
  "title": "Desk",
  "description": "Good condition, minor scratches",
  "category": "furniture",
  "condition": "good",
  "quantity": 1,
  "pickup": {
    "method": "pickup",
    "location": { "type": "Point",
                  "coordinates": [102.8236, 16.4322] },
    "address": "Khon Kaen"
  },
  "expiresAt": "2026-10-20T00:00:00.000Z"
}

Response: 201 Created
```

1.3 ยื่นคำขอรับของ

```
POST /api/v1/listings/:listingId/reservations
Content-Type: application/json

{ "quantity": 1, "message": "I can pick up on Saturday" }

Response: 201 Created
```

ภาคผนวก B

โครงสร้างโปรเจกต์และวิธีการรันระบบ

2.1 โครงสร้างโปรเจกต์

```
src/
  config/        database connection, Swagger config
  controllers/   business logic for each API resource
    web/         thin controllers for server-rendered pages
  services/      shared query/business logic (listing,
                 reservation, category, admin)
  middlewares/   auth, admin, web-auth, csrf, upload,
                 attachLocals, error handler
  models/        User, Category, Listing, Reservation, Report
  routes/        auth, users, categories, listings,
                 reservations, reports, admin, uploads,
                 web (19 web page routes)
  seed.js        seed 7 default categories
frontend/
  views/         EJS templates (pages/, partials/)
  public/
    css/         overrides.css (Semantic UI theme + Kanit font)
    js/          api.service.js, components.js,
                 thai-geo-select.js, pages/*.js
    data/        Thailand province/district/subdistrict data
    uploads/     user-uploaded image files
server.js        entry point
.env             environment variables (not committed)
```

2.2 คำสั่งการติดตั้งและการรันระบบ

```
npm install        install dependencies
npm run seed       insert 7 default categories
npm run dev        start server with auto-reload
```

เมื่อระบบทำงานแล้ว สามารถเข้าดูเอกสาร API ได้ที่ http://localhost:3000/api-docs และตรวจสอบสถานะเซิร์ฟเวอร์ได้ที่ http://localhost:3000/health

ภาคผนวก C

ตาราง Endpoint ของระบบทั้งหมด

ตารางที่ 11: รายการ Endpoint ทั้งหมดของระบบ

วิธี	เส้นทาง	คำอธิบาย
POST	/auth/register	สมัครสมาชิก
POST	/auth/login	เข้าสู่ระบบ
GET	/auth/me	ดูข้อมูลตนเอง
POST	/auth/logout	ออกจากระบบ
PATCH	/auth/password	เปลี่ยนรหัสผ่าน
GET	/users/me	ดูข้อมูลส่วนตัว
PATCH	/users/me	แก้ไขข้อมูลส่วนตัว
DELETE	/users/me	ปิดบัญชี (Soft Delete)
GET	/users/me/listings	ประกาศที่ตนเองลง
GET	/users	รายชื่อผู้ใช้ (ผู้ดูแลระบบ)
GET	/users/:id	ดูข้อมูลผู้ใช้ (ผู้ดูแลระบบ)
PATCH	/users/:id/status	เปิด/ปิดบัญชี (ผู้ดูแลระบบ)
GET	/categories	ดูหมวดหมู่ทั้งหมด
GET	/categories/:id	ดูหมวดหมู่เดียว
POST	/categories	สร้างหมวดหมู่ (ผู้ดูแลระบบ)
PATCH	/categories/:id	แก้ไขหมวดหมู่ (ผู้ดูแลระบบ)
DELETE	/categories/:id	ลบหมวดหมู่ (ผู้ดูแลระบบ)
GET	/listings	ค้นหาประกาศ (หมวดหมู่, ระยะทาง, คำค้น)
GET	/listings/:id	ดูประกาศเดียว
POST	/listings	ลงประกาศ
PATCH	/listings/:id	แก้ไขประกาศ (เจ้าของ/ผู้ดูแลระบบ)
DELETE	/listings/:id	ยกเลิกประกาศ
POST	/listings/:id/reservations	ยื่นคำขอรับของ

วิธี	เส้นทาง	คำอธิบาย
GET	/listings/:id/reservations	ดูรายชื่อผู้ขอ (เจ้าของ)
POST	/listings/:id/reports	รายงานประกาศ
GET	/reservations/my	คำขอของฉัน
GET	/reservations/unread-count	จำนวนคำขอที่ยังไม่อ่าน (Badge แจ้งเตือน)
PATCH	/reservations/read-all	ทำเครื่องหมายอ่านแล้วทั้งหมด
PATCH	/reservations/:id/read	ทำเครื่องหมายอ่านแล้วทีละรายการ
PATCH	/reservations/:id/approve	อนุมัติคำขอ (เจ้าของ)
PATCH	/reservations/:id/reject	ปฏิเสธคำขอ (เจ้าของ)
PATCH	/reservations/:id/cancel	ยกเลิกคำขอ (ผู้ขอ)
PATCH	/reservations/:id/complete	ยืนยันส่งมอบ (เจ้าของ)
GET	/reports	รายการรายงาน (ผู้ดูแลระบบ)
GET	/reports/:id	ดูรายงานเดียว (ผู้ดูแลระบบ)
PATCH	/reports/:id	อัปเดตสถานะรายงาน (ผู้ดูแลระบบ)
GET	/admin/stats	สรุปตัวเลขภาพรวมระบบ (ผู้ดูแลระบบ)
POST	/uploads	อัปโหลดไฟล์รูปภาพ ได้ URL กลับมาใช้ในฟิลด์ images

เส้นทางหน้าเว็บ (EJS + Semantic UI)

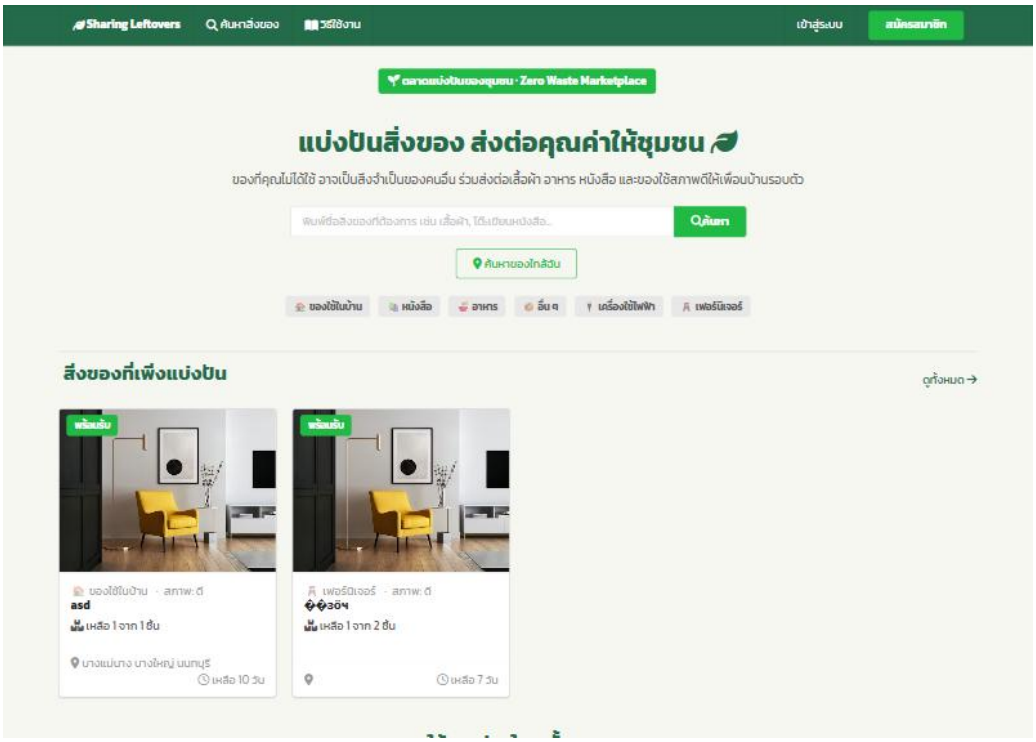
วิธี	เส้นทาง	คำอธิบาย
GET	/	หน้าแรก: Hero ค้นหา หมวดหมู่ และประกาศล่าสุด
GET	/listings	ค้นหาประกาศ (หมวดหมู่ สภาพ ระยะทาง จังหวัด/อำเภอ/ตำบล)
GET	/listings/new	ฟอร์มลงประกาศใหม่ (สมาชิก)
GET	/listings/:id	รายละเอียดประกาศ (ปุ่ม Action ตามบทบาทผู้ใช้)
GET	/listings/:id/edit	แก้ไขประกาศ (ล็อกอัตโนมัติหากมีคำขอรับแล้ว)
GET	/listings/:id/reservations	จัดการคำขอที่ได้รับ (เจ้าของประกาศ)
GET	/auth	เข้าสู่ระบบ/สมัครสมาชิก (รองรับ returnTo)
POST	/auth/logout	ออกจากระบบ
GET	/dashboard	ภาพรวมส่วนตัว (สมาชิก)
GET	/reservations/my	คำขอที่ฉันส่งไป
GET	/users/me/listings	ประกาศทั้งหมดของฉัน
GET	/users/me	โปรไฟล์ของฉัน
GET	/auth/password	เปลี่ยนรหัสผ่าน
GET	/admin	ภาพรวมระบบ (ผู้ดูแลระบบ)
GET	/admin/listings	จัดการประกาศทั้งหมด (ผู้ดูแลระบบ)
GET	/admin/reports	จัดการรายงานปัญหา (ผู้ดูแลระบบ)
GET	/admin/users	จัดการผู้ใช้ (ผู้ดูแลระบบ)
GET	/admin/categories	จัดการหมวดหมู่ (ผู้ดูแลระบบ)
GET	/help/how-it-works	วิธีใช้งานและคำถามที่พบบ่อย

หมายเหตุ: หน้าเว็บทั้งหมดใช้สถาปัตยกรรม SSR Shell + AJAX Form กล่าวคือหน้า GET ข้างต้นแสดงโครงหน้าเปล่าพร้อมข้อมูลเริ่มต้นจากฝั่งเซิร์ฟเวอร์ ส่วนการบันทึกข้อมูล (สมัครสมาชิก ลงประกาศ อนุมัติคำขอ ฯลฯ) เรียกผ่าน REST API ชุดเดียวกับตารางก่อนหน้าโดยตรงด้วย JavaScript ฝั่งไคลเอนต์ (`api.service.js`) จึงไม่มีการเพิ่มเส้นทาง POST/PATCH สำหรับหน้าเว็บแยกต่างหาก

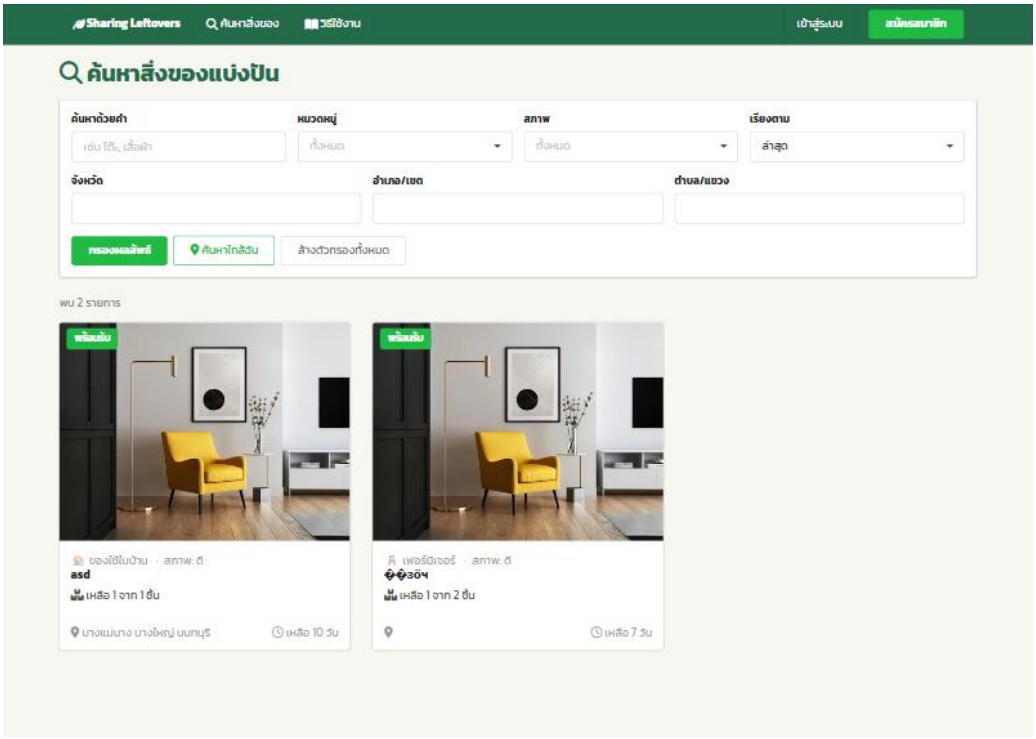
ภาคผนวก D

ภาพหน้าจอระบบ

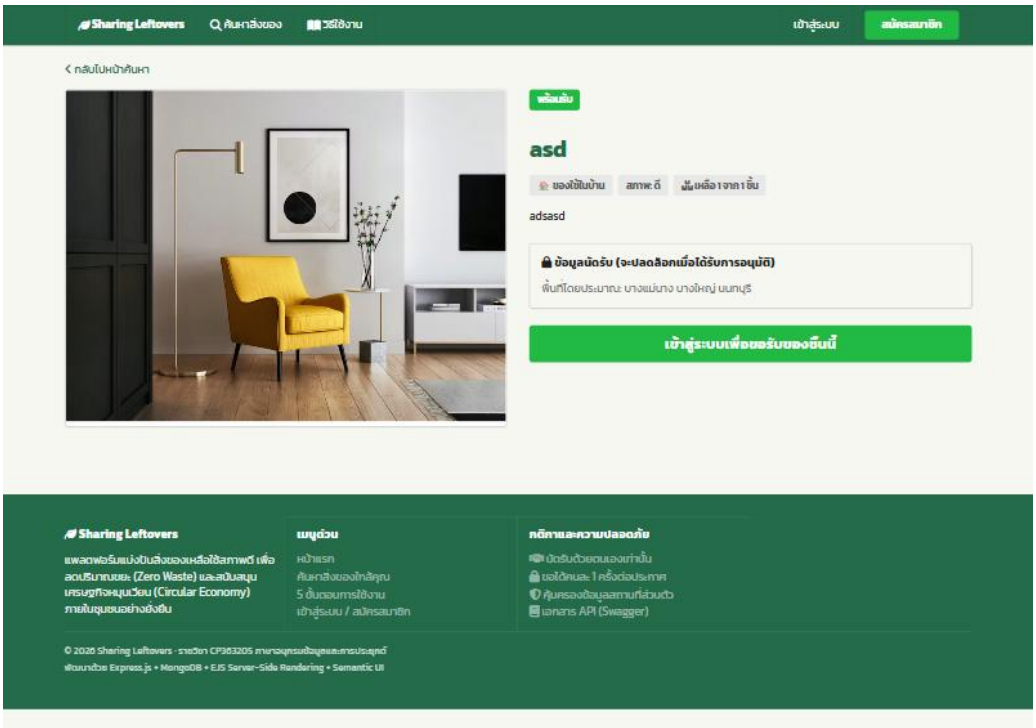
ภาคผนวกนี้แสดงภาพหน้าจอจริงของระบบหลังการพัฒนาแล้วเสร็จ ครอบคลุมทั้งฝั่งผู้เยี่ยมชม สมาชิก เจ้าของประกาศ และผู้ดูแลระบบ โดยหน้าจอทั้งหมดพัฒนาด้วยไลบรารี Semantic UI ร่วมกับไอคอน Font Awesome และฟอนต์ Kanit



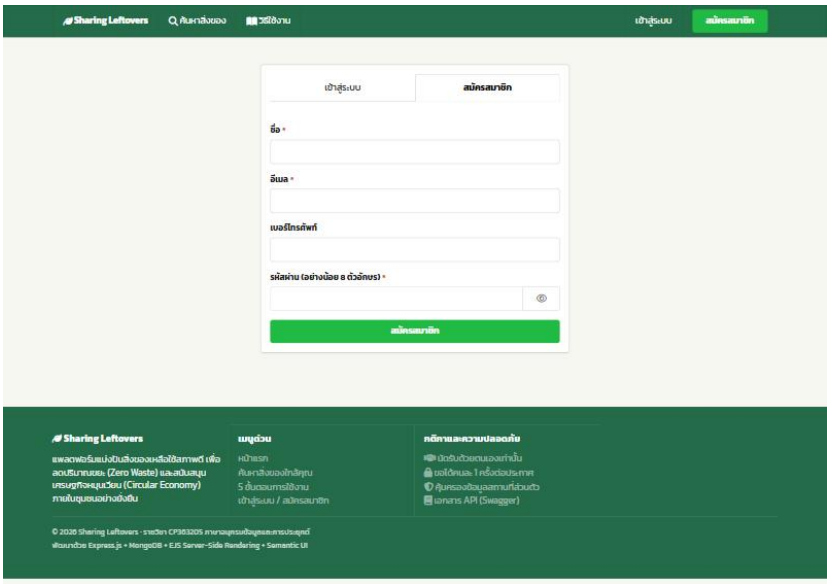
ภาพที่ 3: หน้าแรกของระบบ (มุมมองผู้เยี่ยมชม)



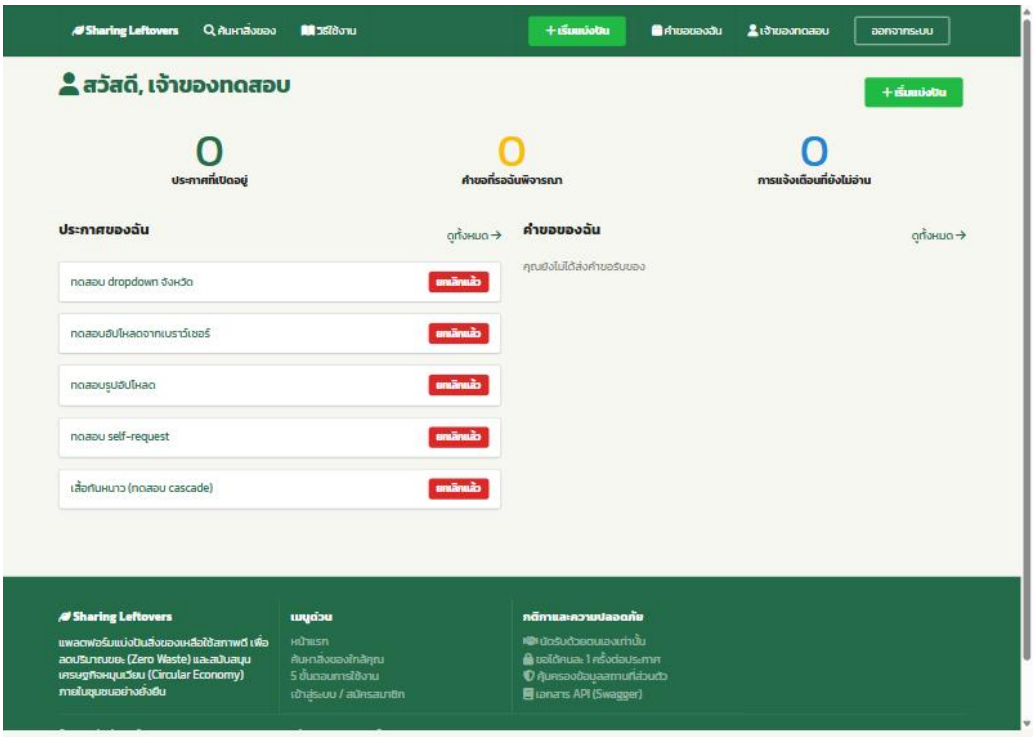
ภาพที่ 4: หน้าค้นหาประกาศ พร้อมตัวกรองหมวดหมู่ สภาพ และพื้นที่



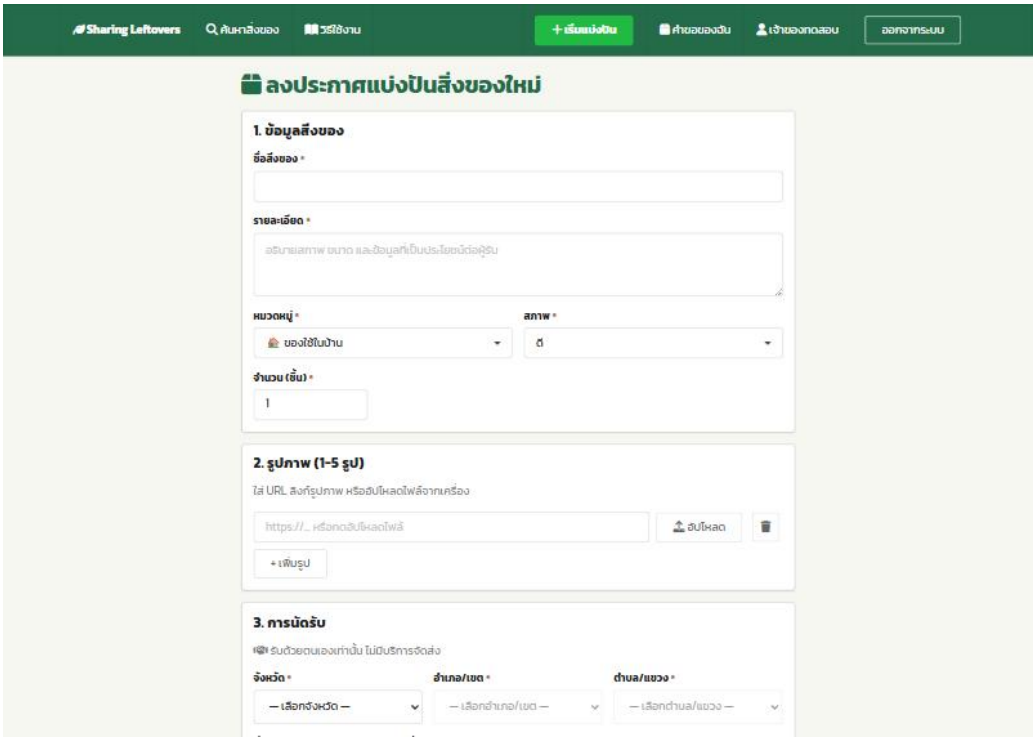
ภาพที่ 5: หน้ารายละเอียดประกาศ (มุมมองผู้เยี่ยมชม — ข้อมูลนัดรับถูกปกปิดจนกว่าจะได้รับอนุมัติ)



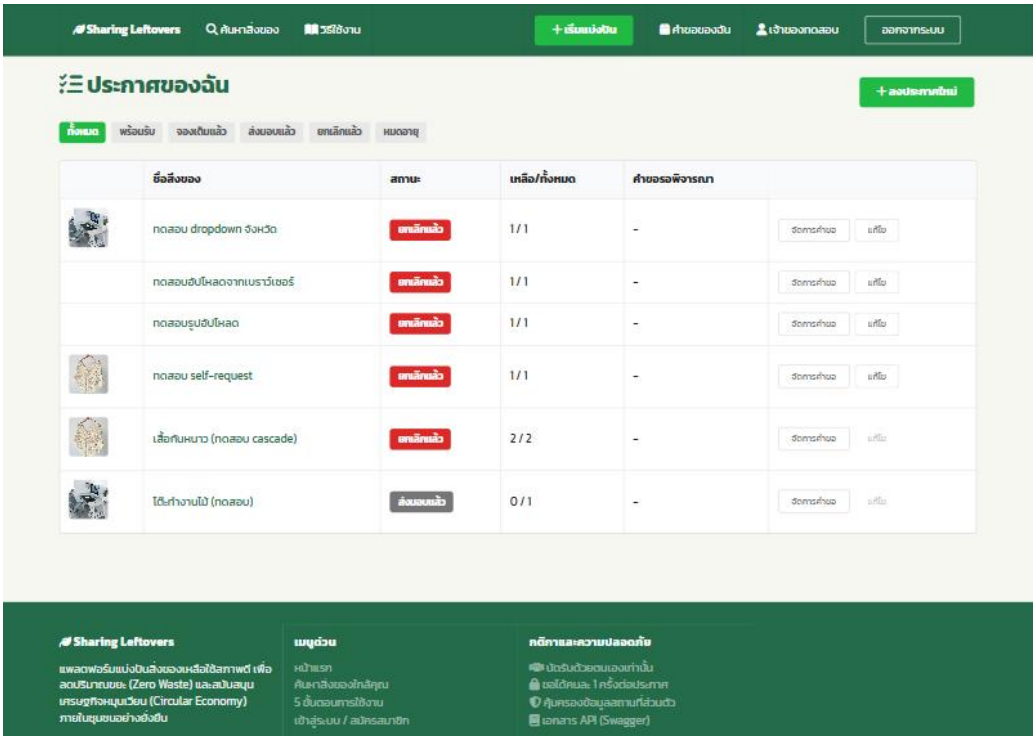
ภาพที่ 6: หน้าเข้าสู่ระบบ/สมัครสมาชิก



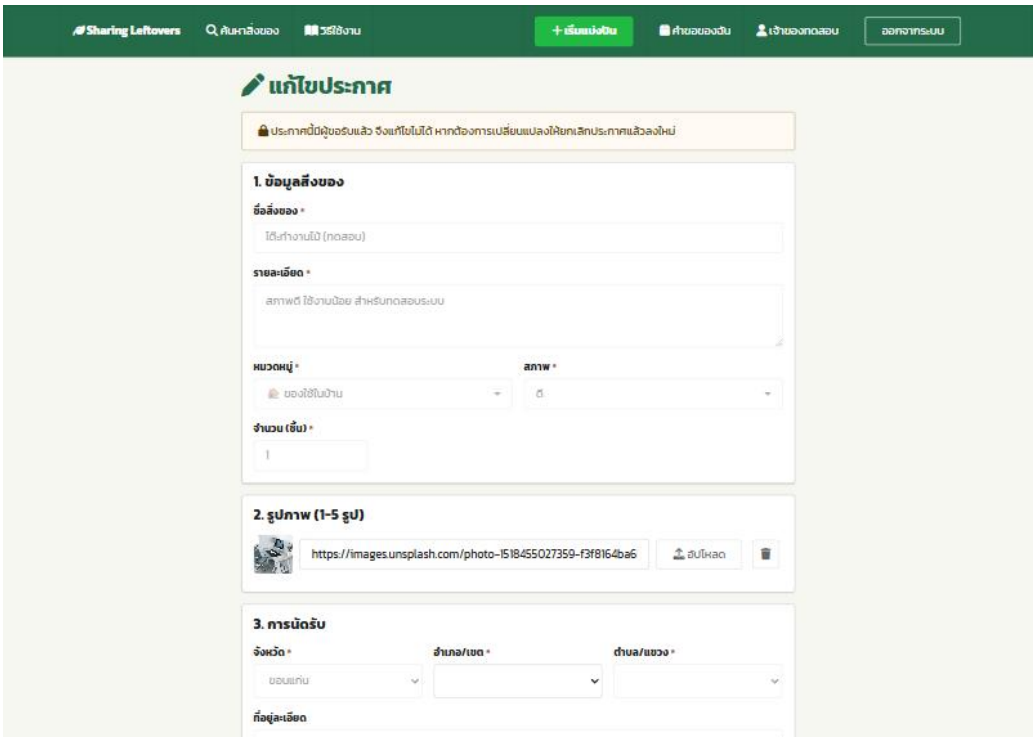
ภาพที่ 7: หน้าภาพรวมส่วนตัว (Dashboard) ของสมาชิก



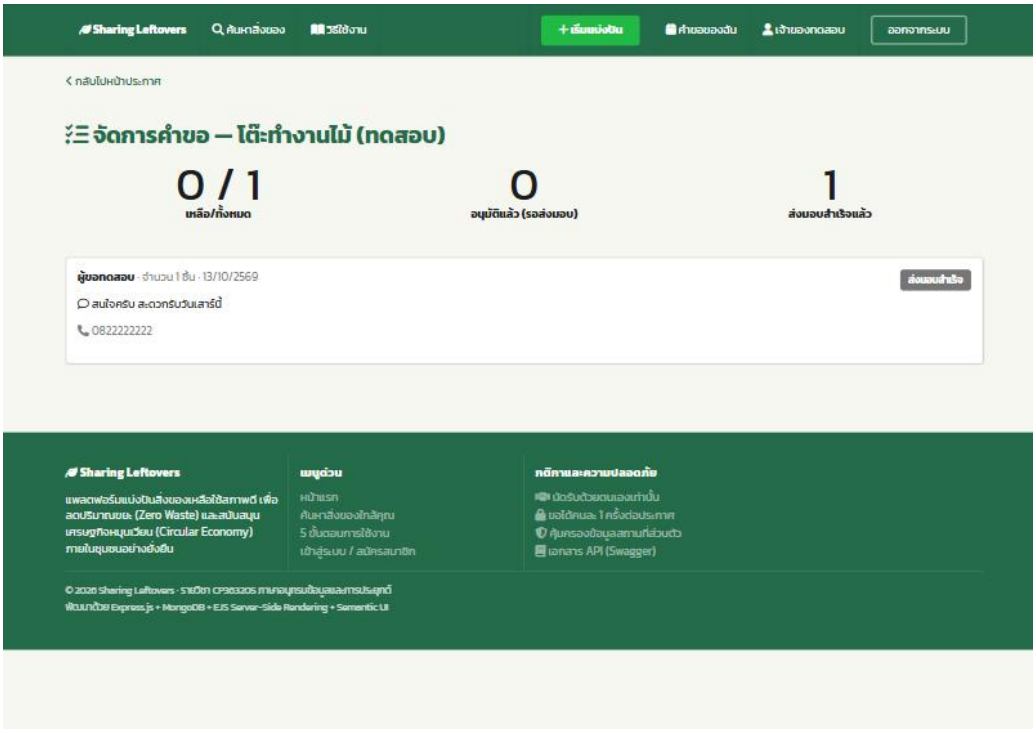
ภาพที่ 8: ฟอร์มลงประกาศใหม่ พร้อม Dropdown จังหวัด/อำเภอ/ตำบลแบบลำดับขั้น และปุ่มอัปโหลดรูปภาพ



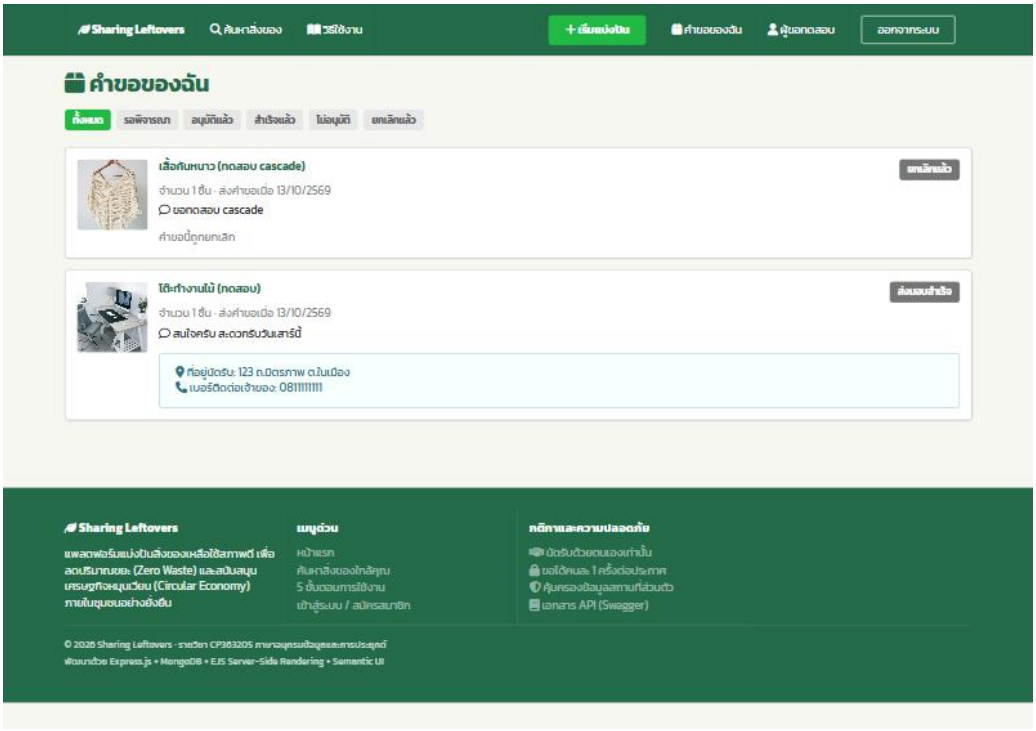
ภาพที่ 9: หน้าประกาศของฉัน แสดงสถานะและจำนวนคำขอที่รอพิจารณาของแต่ละประกาศ



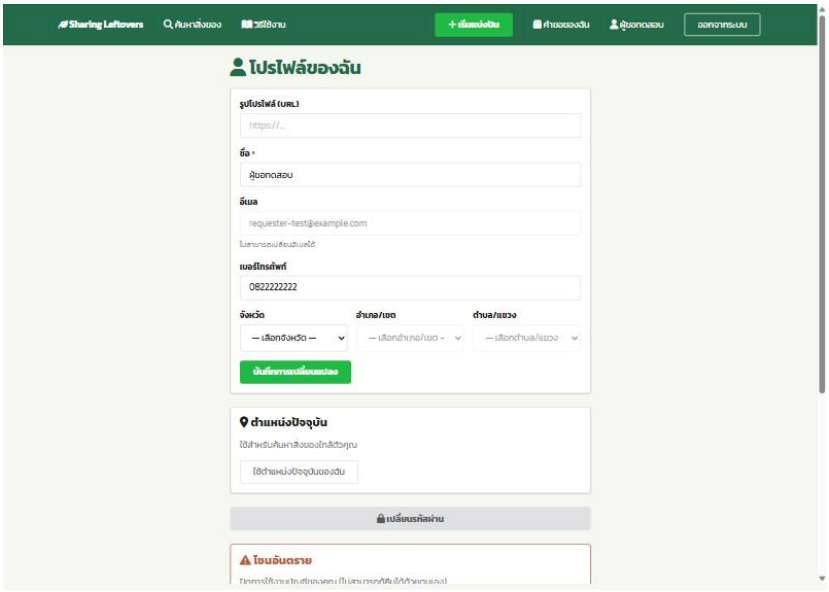
ภาพที่ 10: หน้าแก้ไขประกาศ แสดงสถานะถูกล็อก (Strict Edit Lock) เนื่องจากมีผู้ยื่นคำขอรับแล้ว



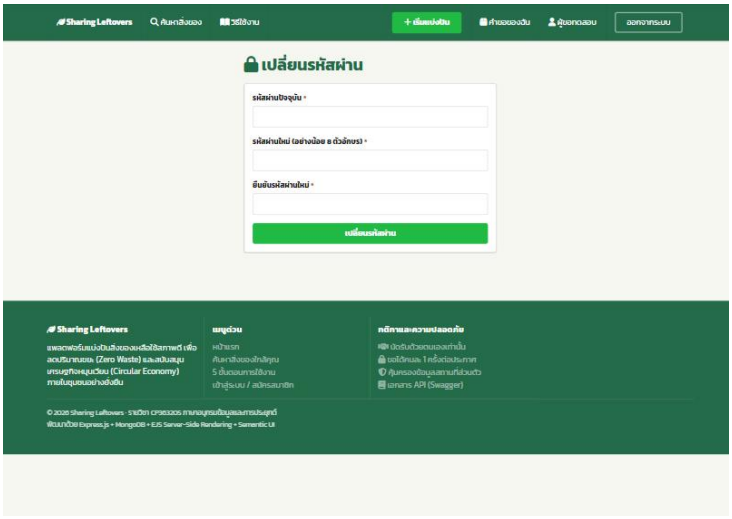
ภาพที่ 11: หน้าจัดการคำขอที่ได้รับ สำหรับเจ้าของประกาศ



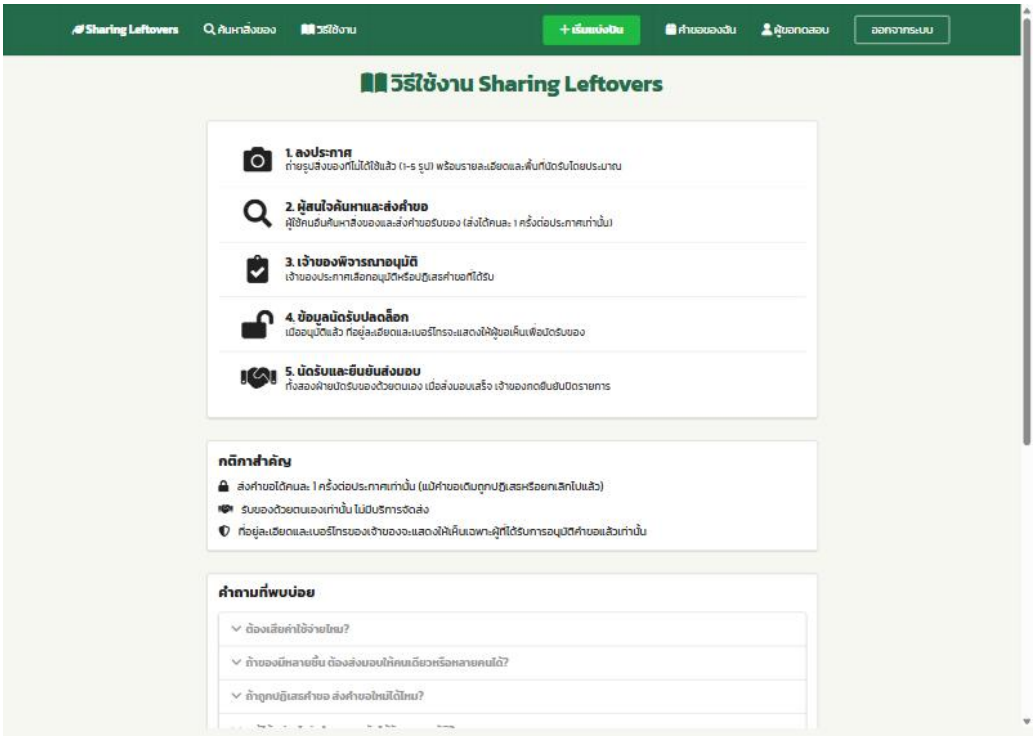
ภาพที่ 12: หน้าคำขอของฉัน แสดงข้อมูลนัดรับที่ปลดล็อกแล้วหลังได้รับอนุมัติ



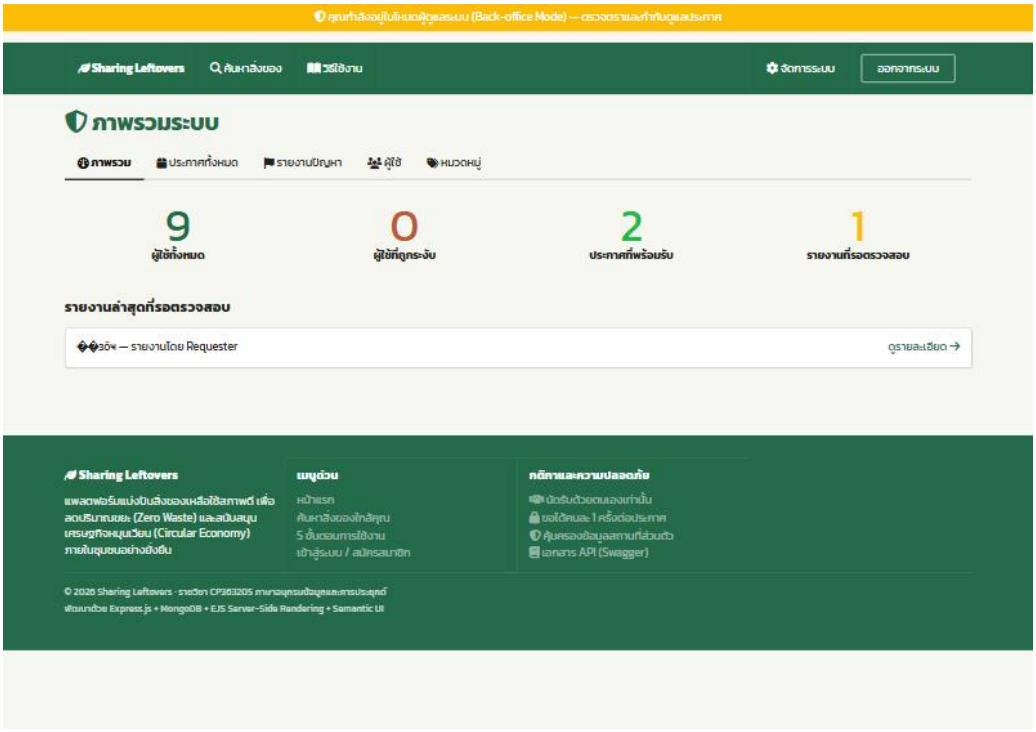
ภาพที่ 13: หน้าโปรไฟล์ของฉัน พร้อม Dropdown ที่อยู่แบบลำดับขั้น



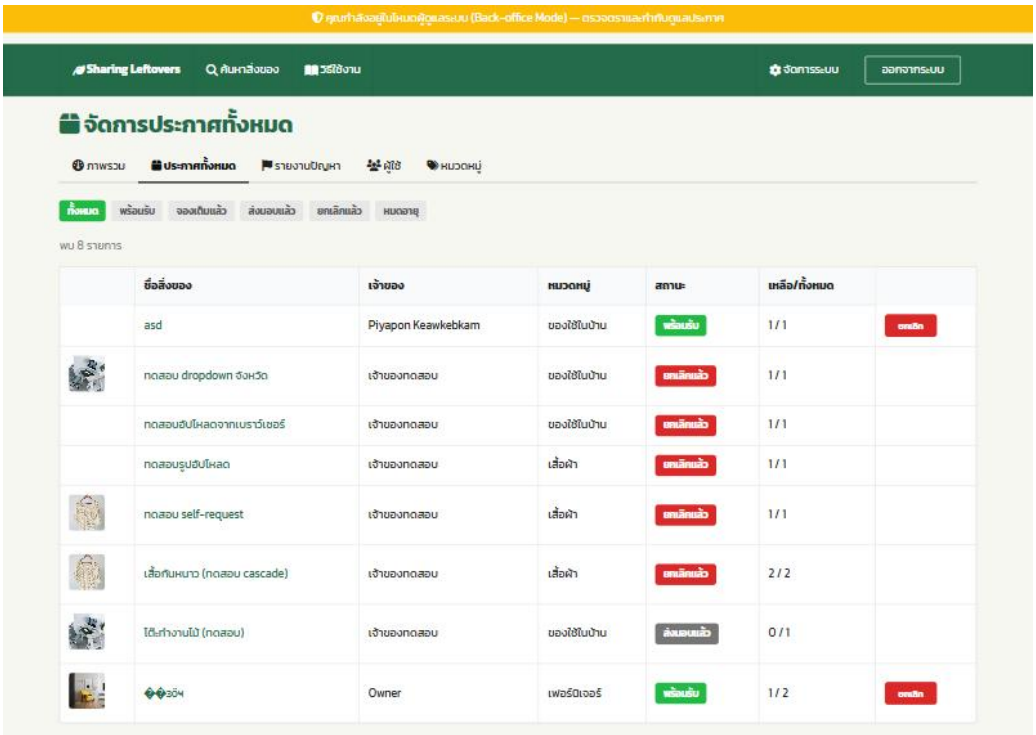
ภาพที่ 14: หน้าเปลี่ยนรหัสผ่าน



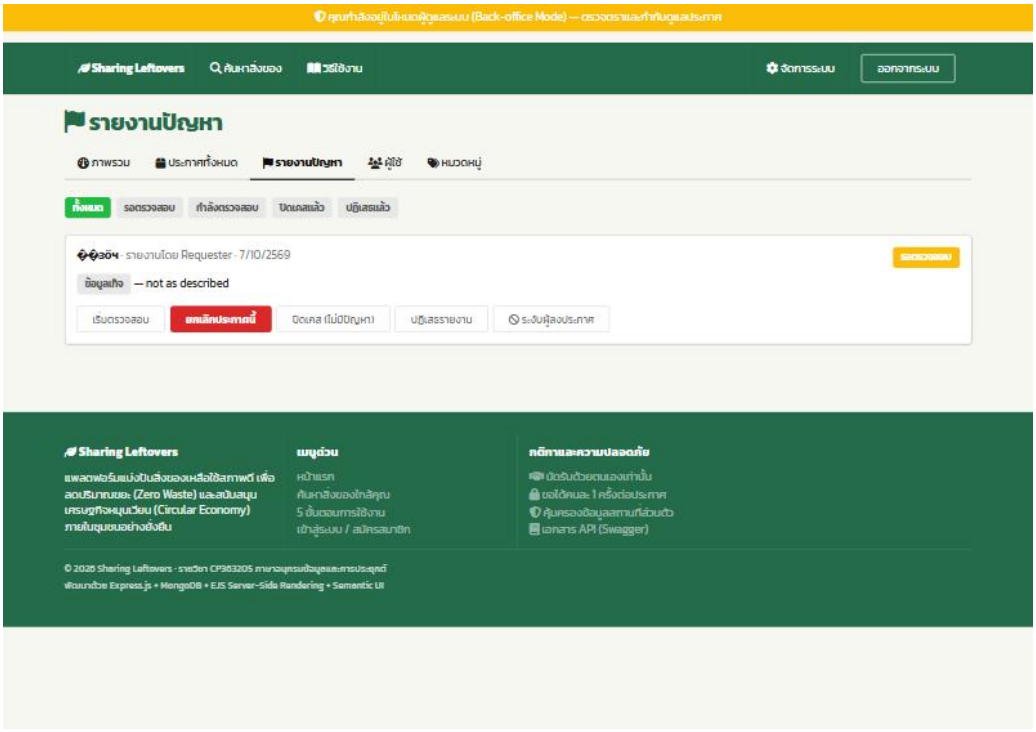
ภาพที่ 15: หน้าวิธีใช้งาน (ขั้นตอนการทำงานและคำถามที่พบบ่อยแบบ Accordion)



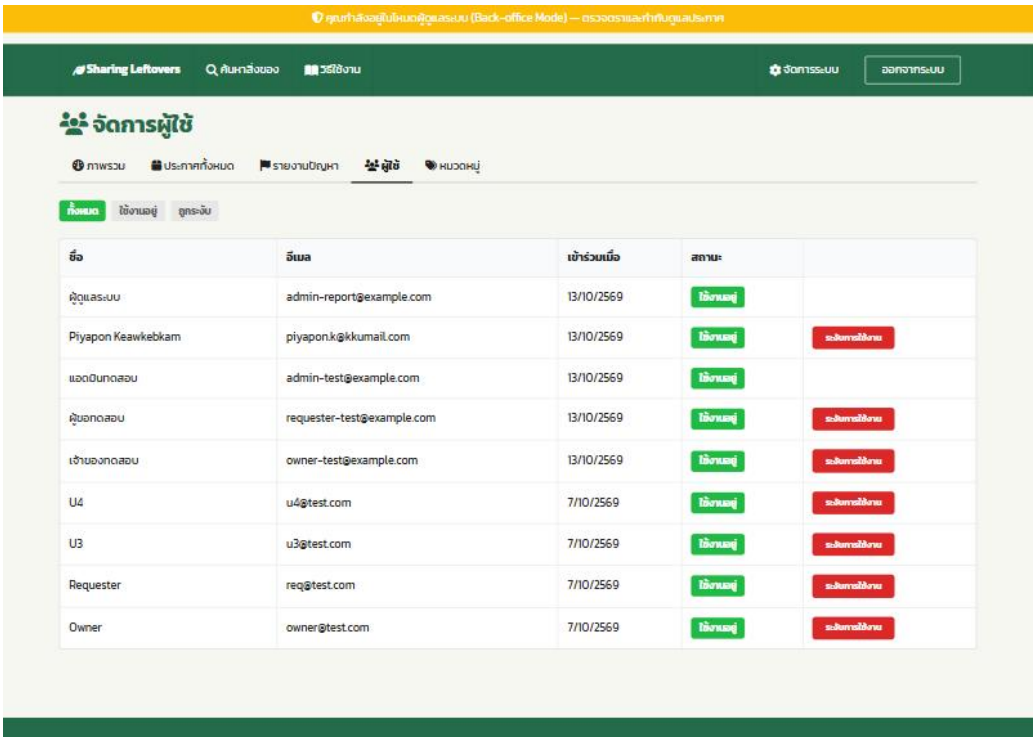
ภาพที่ 16: หน้าภาพรวมระบบสำหรับผู้ดูแลระบบ (Admin Back-office)



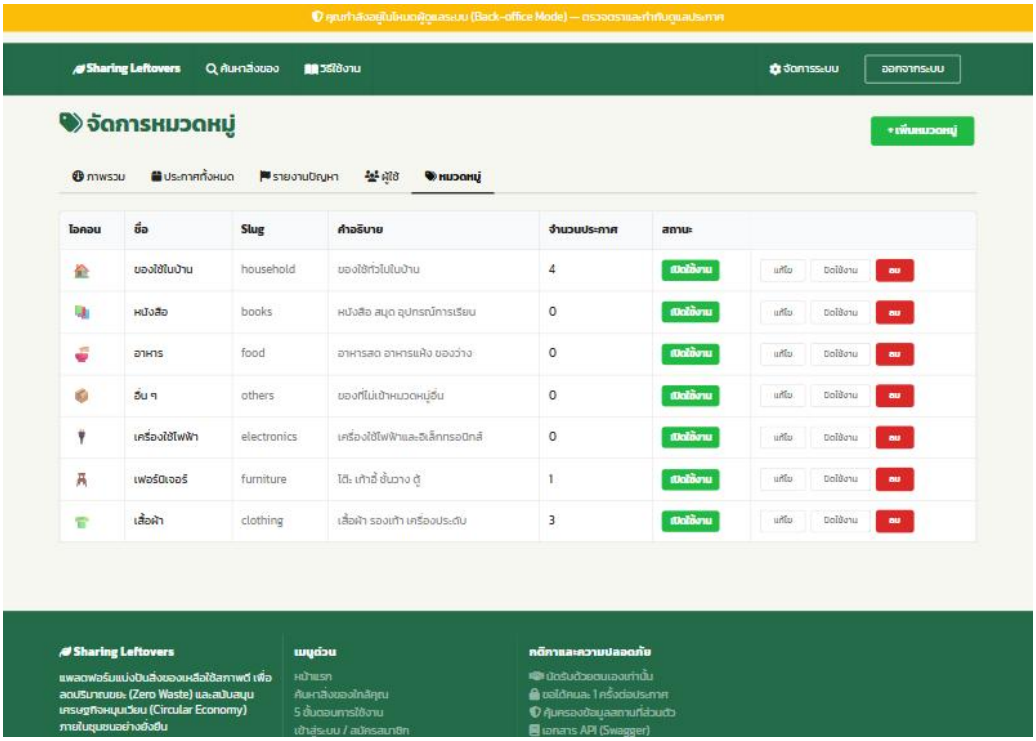
ภาพที่ 17: หน้าจัดการประกาศทั้งหมดของผู้ดูแลระบบ (ดูได้ทุกสถานะรวมถึงที่ยกเลิก/หมดอายุ)



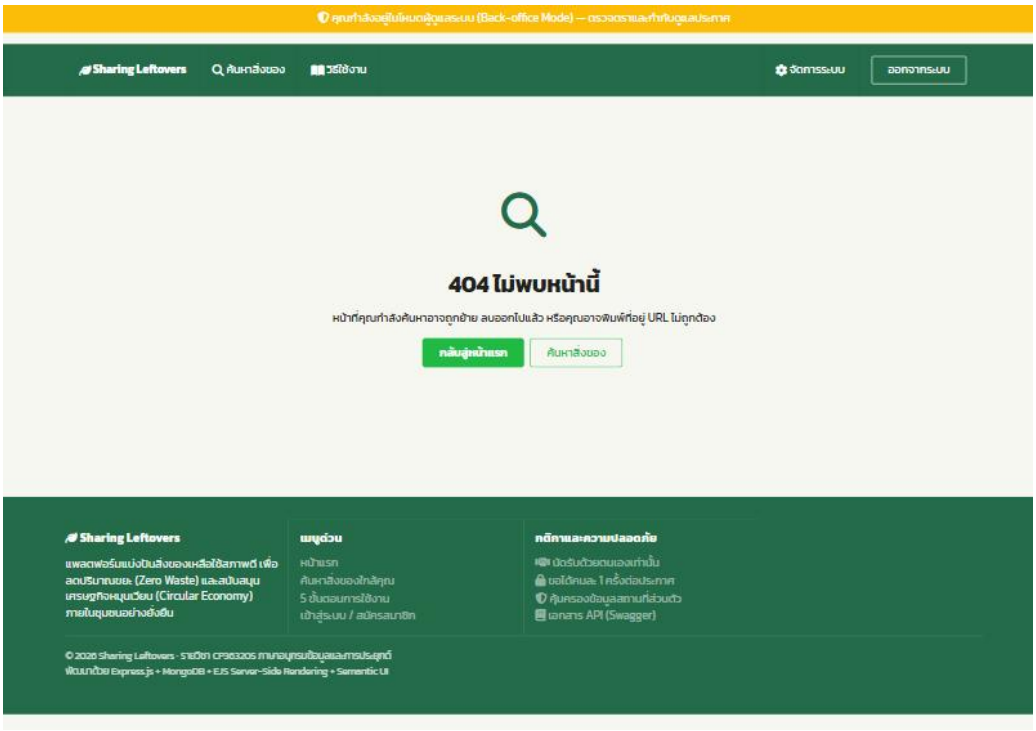
ภาพที่ 18: หน้าจัดการรายงานปัญหาของผู้ดูแลระบบ



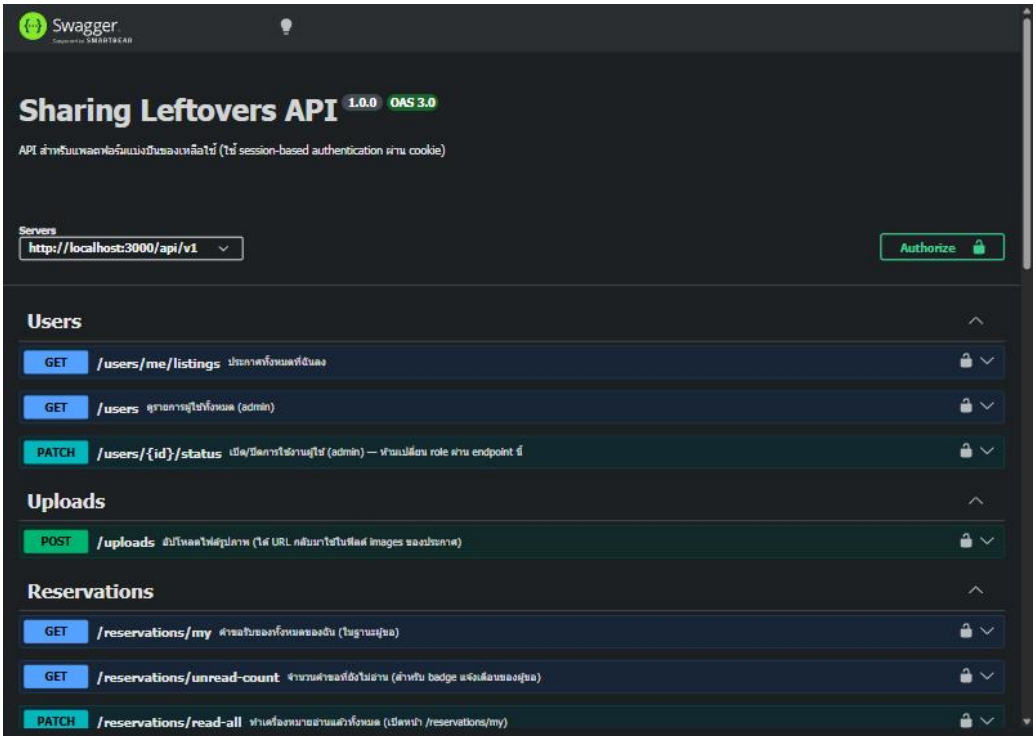
ภาพที่ 19: หน้าจัดการผู้ใช้ของผู้ดูแลระบบ (ระงับ/เปิดการใช้งานบัญชี)



ภาพที่ 20: หน้าจัดการหมวดหมู่ของผู้ดูแลระบบ พร้อมจำนวนประกาศที่ใช้งานแต่ละหมวดหมู่



ภาพที่ 21: หน้าแสดงข้อผิดพลาด 404 เมื่อเข้าถึงเส้นทางที่ไม่มีอยู่จริง



ภาพที่ 22: เอกสาร API ที่สร้างอัตโนมัติผ่าน Swagger UI